

PROVA DE APTIDÃO PROFISSIONAL

H2O ANALYTICS



CURSO PROFISSIONAL DE TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

NOME ALUNO: Tiago Miguel Santos Silva

N.º: 22

PROFESSOR ORIENTADOR: Luís Fernandes

ANO LETIVO: 2025/2026

DEDICATÓRIA

Este projeto representa o culminar de três anos de dedicação, empenho e muito trabalho. Gostaria de expressar o meu sincero agradecimento a todas as pessoas que me apoiaram ao longo deste percurso académico, em especial à minha família e aos meus amigos, que sempre acreditaram no meu potencial e estiveram ao meu lado, sobretudo nesta última etapa, sem dúvida a mais importante e desafiante.

Ao longo deste percurso houve momentos de superação, dificuldades e conquistas, que contribuíram significativamente para o meu crescimento, tanto a nível pessoal como profissional. A todos os que fizeram parte desta caminhada, deixo o meu profundo agradecimento.



AGRADECIMENTOS

Agradecimentos especiais aos professores Luís Fernandes e Ângelo Mateus, pelo excelente apoio e pela colaboração prestada ao longo do desenvolvimento deste projeto.



ÍNDICE

Dedicatória.....	I
Agradecimentos	I
Índice	II
Índice de figuras	IV
Índice de tabelas	V
Lista de siglas e acrónimos	VI
1. Introdução.....	7
1.1. Fundamentação	7
1.2. Descrição do projeto	8
1.3. Objetivos do projeto	9
1.4. Estado do conhecimento.....	9
2. Planificação do projeto	11
2.1. Calendarização.....	11
2.1.1. Calendarização prevista	11
2.1.2. Calendarização realizada	12
2.2. Tecnologia utilizada.....	13
2.3. Protótipo	17
2.4. Base de Dados.....	19
2.5. Componentes eletrónicos.....	21
3. Implementação do projeto	27
3.1. Preparação do ambiente de desenvolvimento	27
3.2. Construção da embarcação e peças 3D	29
3.3. Sistema de propulsão e alimentação	31
3.4. Montagem do circuito eletrónico	32
3.4.1. Controlo da embarcação e configuração do comando.....	32
3.5. Desenvolvimento do Firmware	34
Bibliotecas utilizadas	34
3.5.1. Arquitetura do código	34
3.5.3. Georreferenciação com o SIM808	37



3.5.4. Registo temporal e gravação dos dados.....	39
3.6. Desenvolvimento da Aplicação Web	41
3.6.1. Configuração inicial e base de dados	41
Estrutura da aplicação: rotas, controladores e vistas	41
3.6.2. Autenticação e perfis de utilizador.....	42
3.6.3. Importação e validação dos dados do barco.....	43
3.6.4. Índice de qualidade da água	43
3.6.5. Mapa interativo e visualização	45
3.6.6. Área de administração	46
3.7. Integração e testes.....	47
4. Problemas e soluções encontradas	49
5. Conclusão	51
Referências bibliográficas.....	53

ÍNDICE DE FIGURAS

Figura 1 - Logo do Visual Studio Code	13
Figura 2 - Logo do PlatformIO	13
Figura 3 - Logo do GitHub	13
Figura 4 - Logo da linguagem de programação C/C++	14
Figura 5 - Logo da linguagem de programação PHP	14
Figura 6 - Logo da framework Laravel	14
Figura 7 - Logo Laravel Breeze.....	15
Figura 8 - Logo da aplicação Laragon.....	15
Figura 9 - Logo da Base de dados MySQL.....	15
Figura 10 - Logos das linguagens HTML, CSS e JavaScript	16
Figura 11 - Logo do Tailwind	16
Figura 12 - Logo do Leaflet.js.....	16
Figura 13 - Logo do Chart.js.....	16
Figura 14 - Logo do Claude AI.....	17
Figura 15 - Design do casco do H2O-Maré no Tinkercad.....	18
Figura 16 - Esquema elétrico do H2O-Maré	23
Figura 17 - Esquema elétrico do H2O-Maré Sensores	24
Figura 18 - Esquema elétrico do H2O-Maré GPS & time unit.....	25
Figura 19 - Esquema elétrico do H2O-Maré Unit do Motor	26
Figura 20 - Ambiente de desenvolvimento platform.io	28
Figura 21 - Instalação do php e laravel.....	29
Figura 23 - Casco incompleto da embarcação	30
Figura 22 - Peças da impressão 3d	30
Figura 24 - Cascos da embarcação impressos em 3D	30
Figura 25 - Cascos unidos pela vara de ligação.....	30
Figura 26 - Sondas montadas nos furos laterais do casco.....	31
Figura 27 - Montagem final da embarcação H2O-Maré.....	32
Figura 28 - Recetor FlySky FS-iA6.....	33
Figura 29 - Excerto de código: configuração dos sensores e dos canais RC	35
Figura 30 - Configuração dos sensores.....	36
Figura 31 - GPS controller	38
Figura 32 - Exemplo de uma linha do ficheiro de dados da piscina	40
Figura 33 - Exemplo da Estrutura Laravel.....	42
Figura 34 - Exemplo Mapa de visualização de dados	45
Figura 35 - Screenshot do mapa com marcadores coloridos pelo índice de qualidade.....	46
Figura 36 - Screenshot da página de detalhe de uma localização (gráfico temporal)	46
Figura 37 - Screenshot da área de administração (aprovação de medições)	47
Figura 38 - Teste de navegação na água	48

Figura 39 - Montagem das peças impressas em 3D	50
Figura 40 - Colagem das peças em 3D	50

ÍNDICE DE TABELAS

Tabela 1 - Cronograma previsto.....	11
Tabela 2 - Cronograma realizado	12
Tabela 3 - Lista de componentes utilizados.....	22
Tabela 4 - Ligações ao Arduino Mega	33
Tabela 5 - Parâmetros, intervalos ideais e pesos do índice de qualidade	44

LISTA DE SIGLAS E ACRÓNIMOS

- API — Application Programming Interface (Interface de Programação de Aplicações)
- AT — Attention — conjunto de comandos de texto para controlar módulos de comunicação
- BEC — Battery Eliminator Circuit (circuito regulador de 5 V integrado no ESC)
- DER — Diagrama Entidade-Relacionamento
- DO — Dissolved Oxygen (Oxigénio Dissolvido)
- EC — Electrical Conductivity (Condutividade Elétrica)
- ESC — Electronic Speed Controller (Controlador Eletrónico de Velocidade)
- GNSS — Global Navigation Satellite System (sistema global de navegação por satélite)
- GPRS — General Packet Radio Service (transmissão de dados pela rede móvel)
- GPS — Global Positioning System (Sistema de Posicionamento Global)
- GSM — Global System for Mobile Communications (rede móvel)
- IDE — Integrated Development Environment (Ambiente de Desenvolvimento Integrado)
- JSON — JavaScript Object Notation
- LVC — Low Voltage Cutoff (Corte por Baixa Tensão)
- MySQL — Sistema de gestão de bases de dados relacional
- NiMH — Níquel-Hidreto Metálico (tecnologia de bateria)
- ORM — Object-Relational Mapping (mapeamento objeto-relacional)
- PAP — Prova de Aptidão Profissional
- PHP — Linguagem de programação para desenvolvimento web
- PWM — Pulse Width Modulation (Modulação por Largura de Impulso)
- RC — Remote Control (Controlo Remoto)
- RTC — Real Time Clock (Relógio de Tempo Real)
- TGPSI — Técnico de Gestão e Programação de Sistemas Informáticos

1. INTRODUÇÃO

A PAP (Prova de aptidão profissional) consiste na apresentação e defesa, perante um júri, de um projeto consubstanciado num produto, material ou intelectual, numa intervenção ou numa atuação, consoante a natureza dos cursos, bem como no respetivo relatório final de realização e apreciação crítica, demonstrativo de conhecimentos, aptidões, atitudes e competências profissionais adquiridas ao longo do percurso formativo do aluno, em todas as componentes de formação, com especial enfoque nas áreas de competência inscritas no Perfil dos Alunos à Saída da Escolaridade Obrigatória e no perfil profissional associado à respetiva qualificação.¹

A presente Prova de Aptidão Profissional consiste na apresentação do projeto «H2O Analytics - Sistema de Monitorização Georreferenciada da Qualidade da Água» e do respetivo barco de recolha de dados, «H2O-Maré - Barco de Recolha de Dados Controlado por Rádio Controlo (RC)».

O presente relatório descreve as diferentes fases de desenvolvimento do projeto, desde a conceção da ideia inicial, passando pela pesquisa e análise das soluções atualmente existentes no mercado, pela elaboração do inventário dos componentes necessários à construção do protótipo e pelo processo de desenvolvimento e montagem do mesmo. São ainda abordados os principais desafios e problemas encontrados durante a sua implementação, bem como as soluções adotadas para os ultrapassar.

Por fim, o relatório apresenta os testes realizados em ambiente real, incluindo a colocação do barco na água, a recolha e leitura dos dados obtidos pelos sensores e a análise dos respetivos resultados, permitindo avaliar o desempenho e a eficácia da solução desenvolvida.

1.1. FUNDAMENTAÇÃO

A ideia do projeto H2O Analytics surgiu da observação de que existe uma falta significativa de transparência e de monitorização acessível da qualidade da água em lagos e rios em Portugal. Dados sobre parâmetros como o pH, a condutividade elétrica, o oxigénio dissolvido e a turbidez deveriam ser públicos e facilmente acessíveis aos cidadãos interessados, ambientalistas, pescadores e população em geral, mas, na prática, encontram-se limitados, dispersos ou desatualizados.

¹ Portaria n.º 235-A/2018, de 23 de agosto, que regula os cursos profissionais de nível secundário.

Caso nada seja feito, a degradação da qualidade da água nos corpos hídricos interiores continua a ocorrer sem acompanhamento próximo, o que dificulta a deteção atempada de problemas (poluição, alterações bruscas de parâmetros) e a tomada de decisões por parte de quem usufrui desses espaços. A ausência de dados acessíveis perpetua também a falta de consciencialização ambiental da comunidade.

Beneficiam diretamente desta solução os cidadãos que utilizam estes espaços (pescadores, praticantes de desportos aquáticos e visitantes), as associações ambientais e ainda, num plano mais geral, as entidades locais que pretendam dispor de indicadores simples sobre os seus planos de água. A solução é adequada porque permite recolher dados em tempo quase real em locais específicos, a baixo custo, recorrendo a hardware acessível e a uma aplicação web de consulta simples. Não existe, à data do desenvolvimento, uma solução simples e económica exatamente equivalente no mercado português; as alternativas existentes (estações fixas ou embarcações autónomas) são significativamente mais complexas e dispendiosas.

Em termos de impacto social, o projeto contribui para uma maior transparência ambiental e para a consciencialização da população, ao tornar visíveis e georreferenciados parâmetros que habitualmente são técnicos e pouco divulgados, promovendo assim uma relação mais informada entre as pessoas e os recursos hídricos da sua região.

1.2. DESCRIÇÃO DO PROJETO

O projeto H2O Analytics é composto por duas partes complementares: o barco de recolha de dados e a plataforma web de visualização.

O barco de recolha de dados, designado H2O-Maré, é uma embarcação controlada remotamente (RC), equipada com cinco sensores de qualidade da água — pH, condutividade elétrica (EC), oxigénio dissolvido (DO), turbidez e temperatura — e com um módulo SIM808, que fornece as coordenadas GPS. À medida que navega, o barco recolhe periodicamente as leituras dos sensores e associa-lhes as respetivas coordenadas geográficas e o instante de recolha (fornecido por um relógio de tempo real, RTC), gravando cada registo em formato JSON num ficheiro de texto no cartão MicroSD. O operador conduz o barco através de um comando de radiocontrolo até aos pontos de interesse do plano de água.

A plataforma web de visualização é uma aplicação desenvolvida em Laravel (framework PHP) que lê o ficheiro de dados gerado pelo barco, interpreta-o como JSON, valida os registos e armazena-os numa base de dados MySQL, sujeitando-os a um fluxo de moderação. Os dados aprovados são apresentados num mapa interativo com marcadores georreferenciados, coloridos segundo um índice

de qualidade da água calculado pela plataforma, permitindo ao utilizador consultar as análises por localização em lagos e rios. Desta forma, o sistema resolve o problema do acesso fácil a dados locais de qualidade da água, sendo dirigido sobretudo a adultos interessados nas questões ambientais.

1.3. OBJETIVOS DO PROJETO

O objetivo geral do projeto consiste em desenvolver um sistema de monitorização georreferenciada da qualidade da água em corpos de água interiores, utilizando hardware acessível e uma aplicação web de consulta pública.

Deste objetivo geral decorrem quatro objetivos específicos. O primeiro foi construir e testar um barco RC equipado com sensores de qualidade da água e módulo GPS, capaz de navegar de forma controlada num plano de água. O segundo é implementar o registo de dados geolocalizados em cartão MicroSD, associando a cada medição as coordenadas e o instante de recolha. O terceiro é desenvolver um website em Laravel com base de dados MySQL para a visualização dos dados recolhidos num mapa interativo. Por fim, o quarto objetivo, de natureza transversal, é aplicar e consolidar as competências de programação (C++, PHP, JavaScript, HTML e CSS), de eletrónica e de bases de dados adquiridas ao longo do curso.

1.4. ESTADO DO CONHECIMENTO

Antes de iniciar o desenvolvimento, foi necessário aprofundar um conjunto de conhecimentos teóricos e técnicos que o curso não cobre diretamente. No domínio da qualidade da água, estudaram-se os parâmetros a medir e o seu significado: o pH indica a acidez ou alcalinidade da água; a condutividade elétrica (EC) reflete a salinidade e a mineralização; o oxigénio dissolvido (DO) traduz o nível de oxigénio disponível para a vida aquática; a turbidez mede a clareza da água; e a temperatura, além de ser um parâmetro por si só, influencia a leitura dos restantes.

No domínio da eletrónica, foi necessário compreender o funcionamento, a calibração e a documentação dos sensores analógicos e digitais utilizados, incluindo os protocolos de comunicação 1-Wire, I2C, SPI e UART, bem como os comandos AT, através dos quais o módulo SIM808 é controlado. Estudaram-se ainda os modelos matemáticos de conversão dos valores analógicos dos sensores de EC, de DO e de pH para as respetivas unidades físicas, incluindo a compensação de temperatura, e o armazenamento de dados em cartão MicroSD com Arduino (biblioteca SD), em conjunto com a utilização de um relógio de tempo real (RTC DS3231).

Por fim, no domínio do desenvolvimento web, aprofundou-se a framework Laravel (PHP), nomeadamente o sistema de autenticação Breeze, o middleware de controlo de acessos por perfil, o mecanismo de migrations, factories e seeders para a gestão da base de dados, a integração de mapas interativos com a biblioteca Leaflet e a leitura e validação de ficheiros em formato JSON.

2. PLANIFICAÇÃO DO PROJETO

2.1. CALENDARIZAÇÃO

A primeira etapa consistiu na elaboração de um cronograma com todas as fases do projeto e respetivas durações tendo em conta a data estipulada para entrega do mesmo.

2.1.1. CALENDARIZAÇÃO PREVISTA

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Debate de ideias com a turma									
Pesquisa sobre o tema e preparação do ambiente de desenvolvimento									
Desenho do protótipo (casco + layout)									
Pesquisa sobre o website									
Desenvolvimento do website									
Desenvolvimento de firmware (PlatformIO/C++)									
Desenvolvimento eletrónico (wiring, protótipos)									
Integração e testes									
Deployment do website e inserção de dados através do barco									

Tabela 1- Cronograma previsto

2.1.2. CALENDARIZAÇÃO REALIZADA

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Debate de ideias com a turma									
Pesquisa sobre o tema e preparação do ambiente de desenvolvimento									
Desenho do protótipo (casco + layout)									
Pesquisa sobre o website									
Desenvolvimento do website									
Desenvolvimento de firmware (PlatformIO/C++)									
Desenvolvimento eletrónico (wiring, protótipos)									
Montagem do barco									
Integração e testes									

Tabela 2 - Cronograma realizado

2.2. TECNOLOGIA UTILIZADA

As tecnologias e ferramentas utilizadas foram escolhidas em função dos requisitos das duas componentes do projeto, o firmware do barco e a plataforma web e da experiência adquirida ao longo do curso.

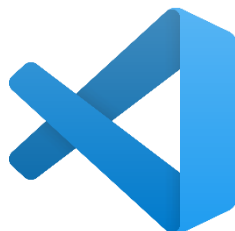


Figura 1 - Logo do Visual Studio Code

O Visual Studio Code foi o editor de código utilizado em ambas as componentes do projeto. Foi escolhido pela sua leveza, pelo vasto ecossistema de extensões e por permitir trabalhar no firmware e na aplicação web dentro do mesmo ambiente de trabalho, por ex: editar texto/código.



Figura 2 - Logo do PlatformIO

O PlatformIO IDE, instalado como extensão do Visual Studio Code, foi o ambiente de desenvolvimento do firmware do Arduino Mega. Foi preferido ao Arduino IDE pela gestão de bibliotecas e de dependências por projeto — declaradas no ficheiro platformio.ini —, pela compilação mais rápida e pela integração direta com o editor.

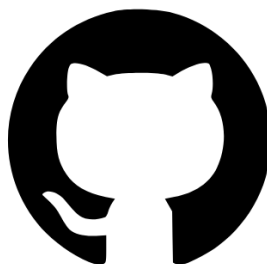


Figura 3 - Logo do GitHub

O GitHub foi utilizado para o controlo de versões e o alojamento do código, em dois repositórios separados, um para o firmware do barco e outro para a aplicação web, permitindo manter o histórico de desenvolvimento e servir de cópia de segurança do projeto.



Figura 4 - Logo da linguagem de programação C/C++

O firmware do barco foi escrito em C++, pela eficiência e pelo acesso de baixo nível ao hardware do microcontrolador, recorrendo a um conjunto de bibliotecas geridas pelo PlatformIO: ArduinoJson para a construção dos registos JSON, SD para o cartão MicroSD, DFRobot_PH e DFRobot_EC10 para a calibração dos sensores de pH e de condutividade, OneWire e DallasTemperature para o sensor DS18B20, RTCLib para o relógio DS3231 e SIM808-EVB para o módulo SIM808.



Figura 5 - Logo da linguagem de programação PHP

O PHP é a linguagem do lado do servidor da aplicação web. Foi escolhida por ser a linguagem de desenvolvimento web lecionada no curso e por ser a base da framework Laravel, utilizada em toda a plataforma.



Figura 6 – Logo da framework Laravel

A framework Laravel estrutura toda a aplicação web: fornece o ORM Eloquent para o acesso à base de dados, o sistema de rotas e de middleware que protege as áreas reservadas e os mecanismos

de migrations e seeders que criam e povoam a base de dados, o que acelerou significativamente o desenvolvimento.

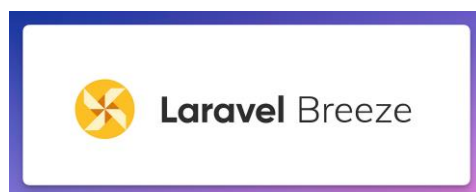


Figura 7 - Logo Laravel Breeze

O Laravel Breeze é o pacote de autenticação oficial da framework, utilizado para implementar o registo, o início de sessão e a recuperação de palavra-passe, servindo de base ao sistema de perfis da plataforma (utilizador, colaborador e administrador).



Figura 8 - Logo da aplicação Laragon

O Laragon foi o ambiente local de desenvolvimento utilizado na componente web, fornecendo o servidor MySQL e simplificando a configuração do PHP e do Laravel durante o desenvolvimento.



Figura 9 - Logo da Base de dados MySQL

O MySQL é o sistema de gestão de bases de dados relacional onde são armazenados os utilizadores, as localizações e as medições recolhidas pelo barco. Foi escolhido por ser o SGBD lecionado no curso e pela integração nativa com o Laravel.



Figura 10 - Logos das linguagens HTML, CSS e JavaScript

O HTML, o CSS e o JavaScript constituem a base da interface web: o HTML estrutura as páginas, o CSS define a apresentação e o JavaScript implementa a interatividade, nomeadamente o mapa e os gráficos.



Figura 11 - Logo do Tailwind

O Tailwind é a framework de CSS utilizada nas vistas da aplicação, permitindo construir uma interface consistente escrevendo as classes de estilo diretamente no HTML, sem folhas de estilo extensas.



Figura 12 - Logo do Leaflet.js

O Leaflet.js é a biblioteca JavaScript open-source que constrói o mapa interativo da plataforma, no qual cada localização é apresentada como um marcador colorido pelo índice de qualidade da água, alimentado por uma API interna em formato GeoJSON.



Figura 13 - Logo do Chart.js

O Chart.js é a biblioteca JavaScript utilizada nos gráficos temporais da página de detalhe de cada localização, permitindo analisar a evolução dos parâmetros ao longo do tempo.



Figura 14 – Logo do Tinkercad

O Tinkercad foi a ferramenta de modelação 3D utilizada para visualizar e preparar o modelo do casco antes da impressão.



Figura 14 - Logo do Claude AI

Ao longo do desenvolvimento deste projeto, recorri à assistência da IA Claude (Anthropic) como ferramenta de apoio, nomeadamente na estruturação e formatação do relatório de PAP e para me ajudar no website a corrigir e a ajudar com problemas de sintaxe e lógicos, mantendo sempre a autoria e as decisões de implementação da plataforma.

2.3. PROTÓTIPO

O protótipo do barco assenta num casco de catamarã impresso em 3D. Por restrições de tempo, optou-se conscientemente por não desenhar o casco de raiz — uma decisão de engenharia que permitiu concentrar o esforço na eletrónica e no software, que constituem o núcleo do projeto. Foi utilizado um design 3D de catamarã já testado e validado, criado por «electrosync» (nome pelo qual o autor do design se identifica no YouTube), o que permitiu poupar tempo e reduzir os custos de prototipagem.

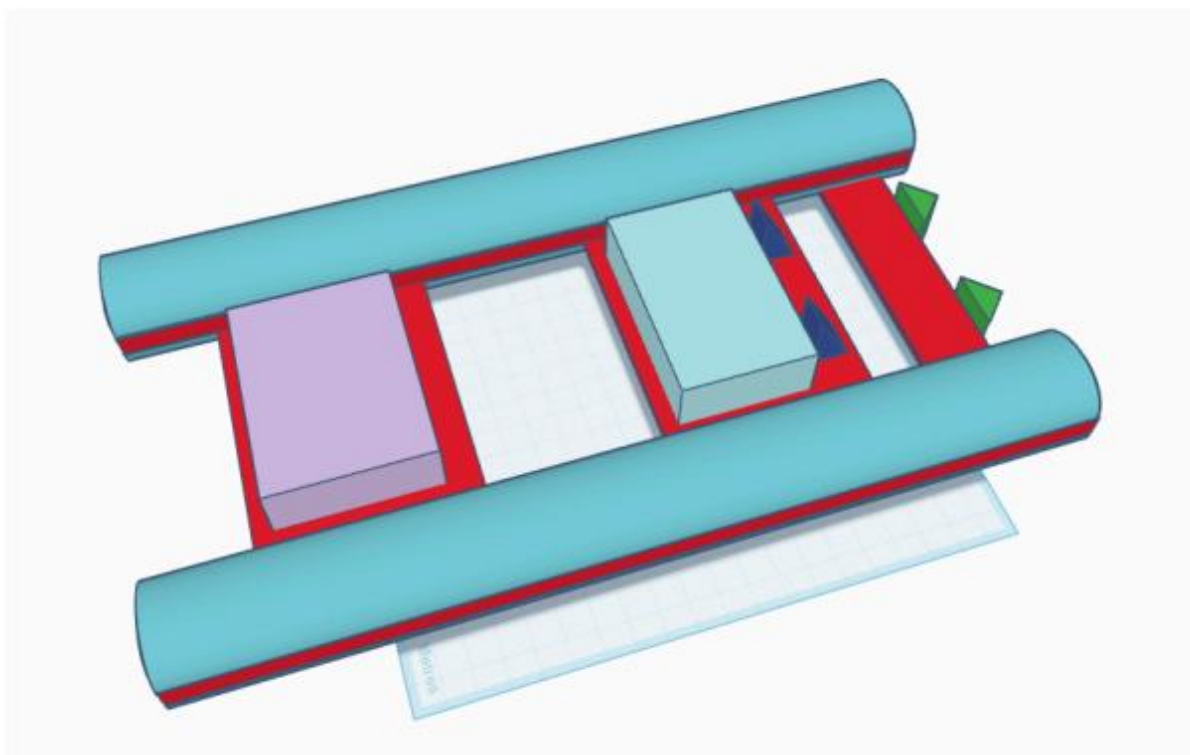


Figura 15 - Design do casco do H2O-Maré no Tinkercad

2.4. BASE DE DADOS

A informação recolhida pelo barco é armazenada e organizada numa base de dados relacional, que constitui o suporte de toda a aplicação web. Antes de apresentar a estrutura das tabelas, importa enquadrar o modelo de dados: o sistema foi desenhado em torno das medições, a entidade central, e das entidades que lhe dão contexto, nomeadamente os utilizadores que as submetem e as localizações onde são recolhidas. Descrevem-se em seguida essa estrutura e as relações entre as tabelas, ilustradas pelo Diagrama Entidade-Relacionamento (DER).

A tabela `measurements` é o núcleo do sistema: cada linha corresponde a uma leitura efetuada pelo barco e guarda os valores dos cinco parâmetros (`ph_sensor`, `ec_sensor`, `dissolved_oxygen`, turbidez e temperatura), as coordenadas geográficas (latitude e longitude, com precisão decimal adequada a coordenadas GPS), o instante de recolha fornecido pelo RTC (`timestamp_sensor`) e ainda o registo original em bruto (`raw_data`, em formato JSON), que permite auditar ou reprocessar os dados caso a lógica de conversão seja alterada. Cada medição guarda também um campo de estado (`status`) e os campos `approved_by`, `approved_at`, `rejected_by` e `rejected_at`, que implementam um fluxo de moderação: as medições submetidas ficam pendentes até serem aprovadas ou rejeitadas por um administrador, garantindo que apenas dados validados são apresentados publicamente no mapa.

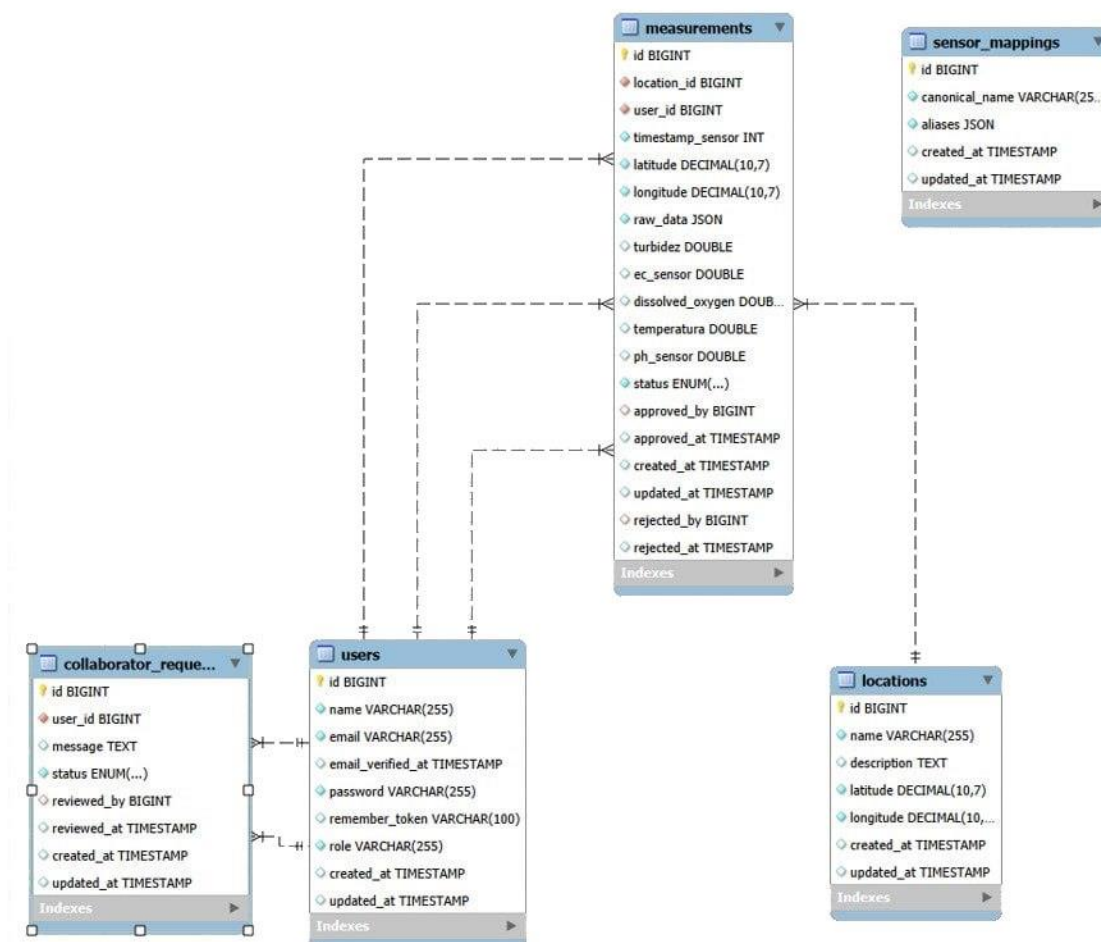


Figura 16 — Diagrama Entidade-Relacionamento (DER) da base de dados

A tabela `users` guarda os utilizadores da plataforma (nome, e-mail e palavra-passe encriptada), incluindo um campo `role` que distingue os três perfis — utilizador autenticado, colaborador e administrador. A tabela `locations` representa as massas de água (nome, descrição e coordenadas de referência), permitindo agrupar medições por local. A tabela `sensor_mappings` guarda o nome canónico de cada sensor e uma lista de aliases em JSON, o que permite normalizar os nomes dos campos vindos do ficheiro do barco mesmo que a nomenclatura do firmware venha a mudar. Por fim, a tabela `collaborator_requests` regista os pedidos dos utilizadores para se tornarem colaboradores, com o estado do pedido e a identificação de quem o reviu.

Quanto às relações: cada medição pertence a um utilizador (`measurements.user_id` → `users.id`) e pode pertencer a uma localização (`measurements.location_id` → `locations.id`), numa relação de um-para-muitos em ambos os casos — um utilizador pode submeter muitas medições e uma localização pode agregar muitas medições. Os campos `approved_by` e `rejected_by` são igualmente chaves

estrangeiras para users, identificando o moderador. A tabela `collaborator_requests` relaciona-se com users tanto pelo autor do pedido (`user_id`) como pelo revisor (`reviewed_by`). Completam o esquema as tabelas de suporte geradas pelo próprio Laravel (`sessions`, `password_reset_tokens`, `migrations`, `cache_locks` e `failed_jobs`), responsáveis pela gestão de sessões, recuperação de palavras-passe e controlo interno da framework.

2.5. COMPONENTES ELETRÓNICOS

O sistema eletrónico do barco é constituído por um microcontrolador, cinco sensores de qualidade da água, três módulos auxiliares e o conjunto de propulsão e radiocontrolo. Descreve-se em seguida a função de cada componente no projeto.

O Arduino Mega 2560 é o cérebro do sistema. Foi escolhido em detrimento do Arduino Uno pelo maior número de entradas analógicas — o projeto utiliza dez apenas entre sensores e canais RC —, pelas múltiplas portas série por hardware (necessárias para o módulo SIM808) e pela maior memória de programa, que acomoda as bibliotecas de todos os periféricos.

Dos cinco sensores, quatro são analógicos da gama Gravity da DFRobot: o sensor de pH (SEN0161-V2) mede a acidez ou alcalinidade da água; o sensor de condutividade elétrica (DFR0300, com constante de célula $K=1$) mede a capacidade da água para conduzir corrente elétrica, indicador de salinidade e mineralização; o sensor de oxigénio dissolvido (SEN0237) mede o oxigénio disponível para a vida aquática; e o sensor de turbidez (SEN0189) mede a clareza da água através da atenuação de um feixe de luz. O quinto sensor, o DS18B20, é um sensor digital de temperatura estanque que comunica pelo protocolo 1-Wire; além de fornecer o parâmetro temperatura, as suas leituras são usadas na compensação térmica dos sensores de EC, DO e pH, cujas medições variam com a temperatura da água.

Os módulos auxiliares dão contexto a cada medição. O módulo SIM808 combina num único componente um recetor GNSS e um modem GSM/GPRS: neste projeto é utilizado para obter as coordenadas geográficas, mas a componente GSM abre a porta, sem hardware adicional, à transmissão de dados em tempo real identificada como melhoria futura. O RTC DS3231 fornece a data e a hora com elevada precisão, mesmo com o sistema desligado, graças à sua bateria de backup. O leitor de cartão MicroSD, ligado por SPI, permite gravar localmente todos os registos.

O conjunto de propulsão e controlo é composto pelo recetor FlySky FS-iA6 de seis canais, que recebe os comandos do operador, por um motor brushed fornecido em conjunto com as hélices, comandado por um ESC HobbyWing SeaKing 30A — variador náutico com refrigeração a água — que controla a velocidade, por um servo que aciona o leme de direção e por uma bateria NiMH de

7,2 V e 5000 mAh. A alimentação da eletrónica de 5 V é assegurada por um conversor step-down ligado à bateria, separando o circuito de potência do circuito lógico.

Componente	Função no projeto	Qtd.	Preço
Arduino Mega 2560	Microcontrolador central: lê os sensores, processa e grava os dados	1	18 €
Sensor de pH (DFRobot SEN0161-V2)	Medição da acidez/alcalinidade da água	1	36 €
Sensor de EC (DFRobot DFR0300, K=1)	Medição da condutividade elétrica	1	64 €
Sensor de DO (DFRobot SEN0237)	Medição do oxigénio dissolvido	1	155 €
Sensor de turbidez (DFRobot SEN0189)	Medição da clareza da água	1	9 €
Sensor DS18B20 (estanque)	Temperatura da água (compensação de EC, DO e pH)	1	7 €
Módulo SIM808 (GPS + GSM/GPRS)	Coordenadas geográficas; GSM disponível para expansão futura	1	25 €
RTC DS3231	Data e hora de cada registo	1	5 €
Leitor de cartão MicroSD	Armazenamento local dos dados	1	3 €
Recetor FlySky FS-iA6 + comando	Radiocontrolo (6 canais)	1	45 €
Motor brushed (fornecido com as hélices)	Propulsão	1	12 €
ESC HobbyWing SeaKing 30A	Controlo de velocidade do motor (variador náutico)	1	40 €
Servo de direção	Controlo do leme	1	8 €
Bateria NiMH 7,2 V 5000 mAh	Alimentação do sistema	1	25 €
Conversor step-down 12 V → 5 V	Alimentação regulada da eletrónica de 5 V	1	4 €
Casco de catamarã impresso em 3D	Estrutura flutuante da embarcação	1	€ 240

Tabela 3 - Lista de componentes utilizados

O custo total da eletrónica rondou os 456 € (preços de mercado de 2026, arredondados), aos quais acresce o custo da impressão 3D do casco pela empresa externa. (240euros).

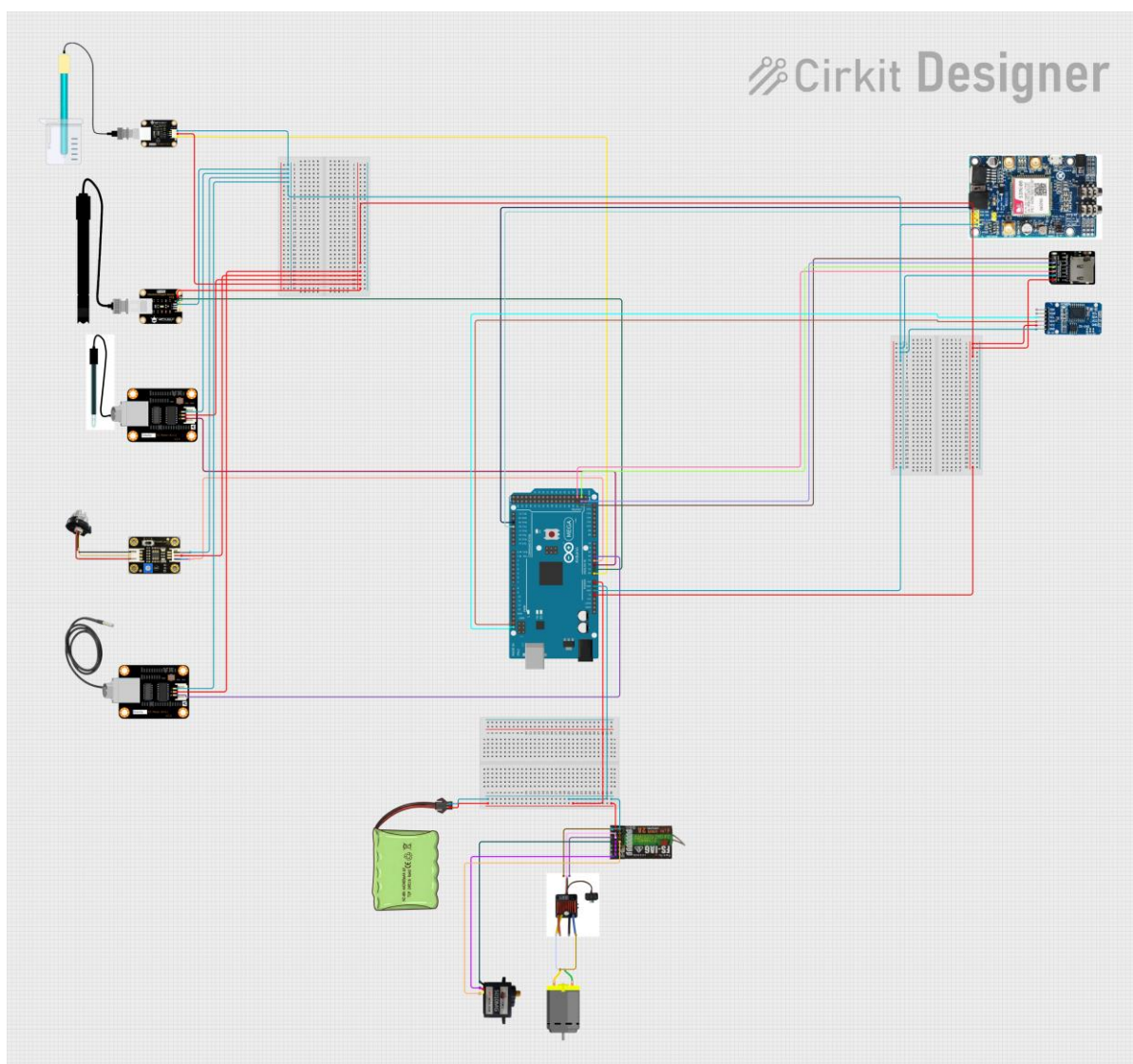


Figura 16 - Esquema elétrico do H2O-Maré

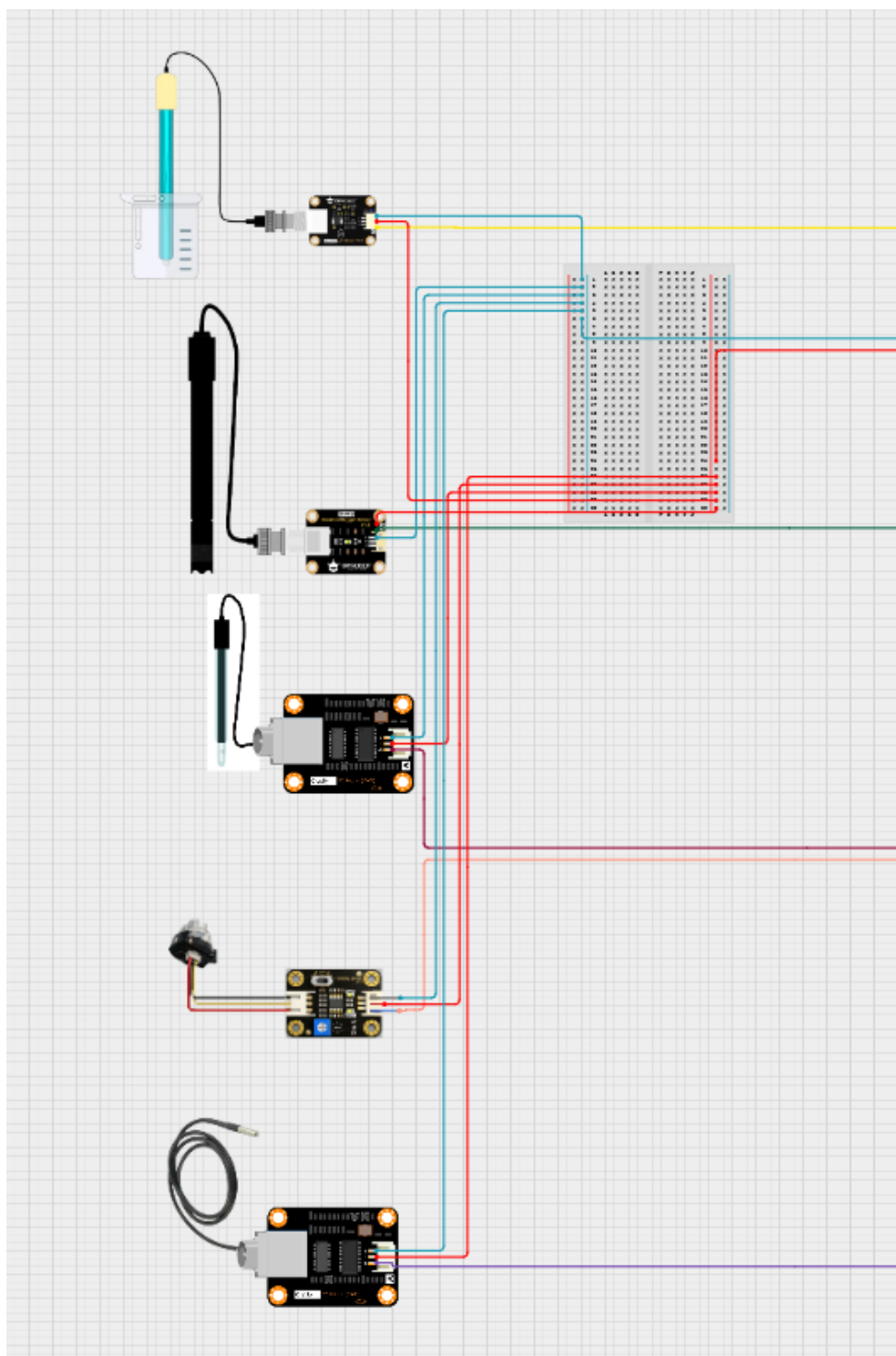


Figura 17 - Esquema elétrico do H2O-Maré Sensores

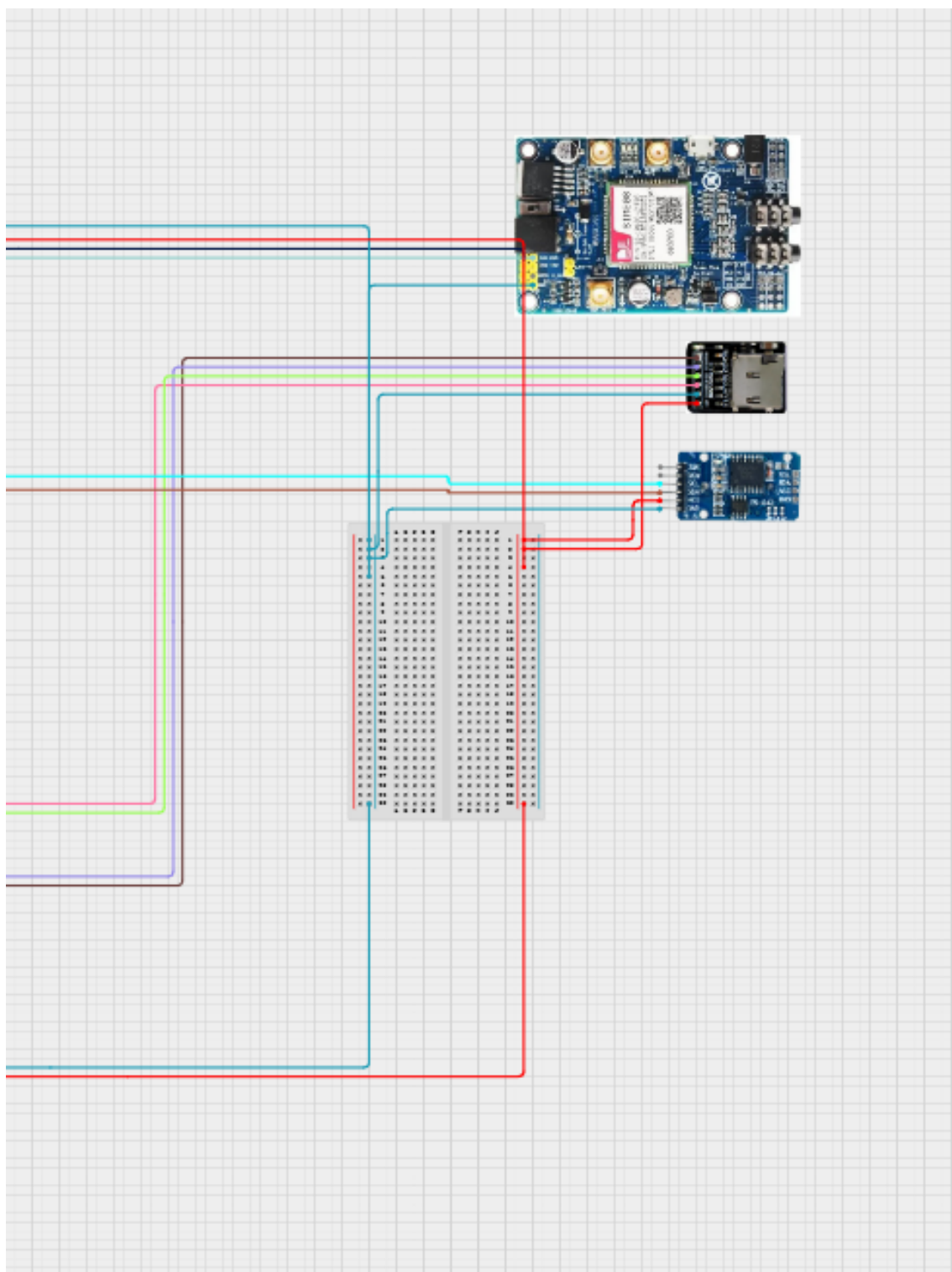


Figura 18 - Esquema elétrico do H2O-Maré GPS & time unit

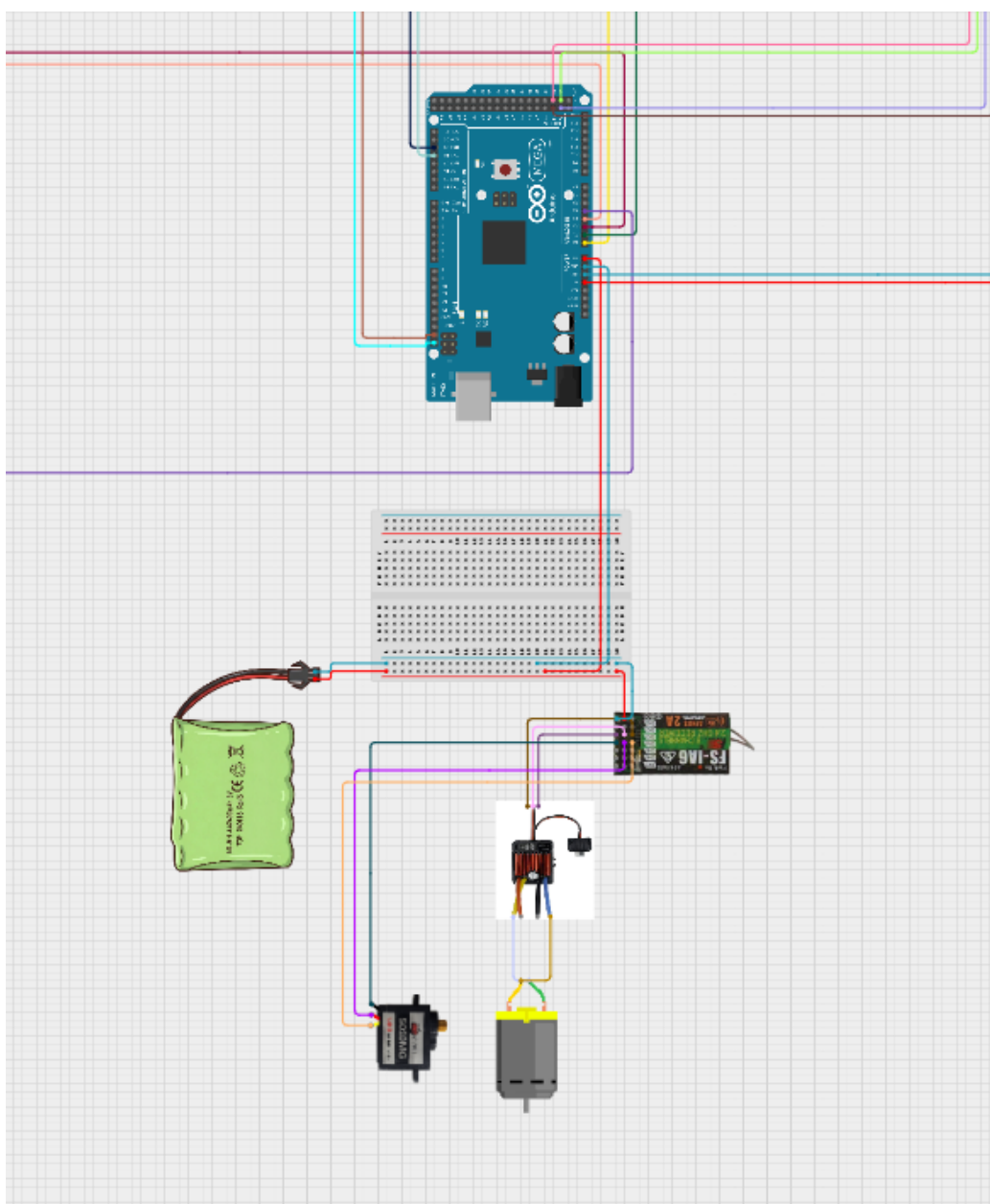


Figura 19 - Esquema elétrico do H2O-Maré Unit do Motor

3. IMPLEMENTAÇÃO DO PROJETO

A implementação do projeto desenvolveu-se em duas frentes paralelas, o subsistema de hardware/firmware (barco) e a aplicação web, que foram posteriormente integradas. Este capítulo descreve, por ordem cronológica de trabalho, a preparação do ambiente de desenvolvimento, a construção da embarcação, o sistema de propulsão e alimentação, a montagem do circuito eletrónico, o desenvolvimento do firmware, o desenvolvimento da aplicação web e, por fim, a integração e os testes do sistema completo.

Ao longo deste capítulo procura-se documentar, passo a passo, todas as etapas do desenvolvimento — desde a preparação das ferramentas até aos testes finais —, de modo a que a leitura permita compreender e, em princípio, reproduzir o percurso seguido. Sempre que relevante, o texto é acompanhado de excertos de código comentados e de imagens das várias fases.

3.1. PREPARAÇÃO DO AMBIENTE DE DESENVOLVIMENTO

O primeiro passo consistiu em preparar o ambiente de trabalho. Instalou-se o Visual Studio Code como editor principal e, sobre ele, a extensão PlatformIO IDE, criando-se o projeto de firmware com a placa Arduino Mega 2560 como alvo. As bibliotecas necessárias, ArduinoJson, SD, DFRobot_PH, DFRobot_EC10, OneWire, DallasTemperature, RTCLib e SIM808-EVB foram declaradas no ficheiro platformio.ini, o que garante que as dependências são geridas por projeto e que a compilação é reprodutível em qualquer máquina.

```

14  const int CS_PIN = 53;
15
16  static constexpr unsigned long INTERVALO_LEITURA = 5000;
17
18  const char* LOG_ARQUIVO = "dados.txt";
19
20  FlyskyController::rc_info rc_config[] = {
21      { "CH5", A8, RC_ATIVO },
22      { "CH2", A9, RC_INATIVO },
23      { "CH3", A10, RC_INATIVO },
24      { "CH4", A11, RC_INATIVO },
25      { "CH5", A12, RC_INATIVO },
26      { "CH1", A13, RC_INATIVO },
27  };
28
29  Sensor::sensor_info config[] = {
30      { "Dissolved Oxygen Sensor", A8, DO_SENSOR, ATIVO },
31      { "EC Sensor", A1, EC_SENSOR, ATIVO },
32      { "Ph Sensor", A2, Ph_SENSOR, ATIVO },
33      { "Temperatura", A3, TEMPERATURA, ATIVO },
34      { "Turbidez", A4, TURBIDEZ, ATIVO },
35  };
36
37  const int NUM_CANAIS_RC =
38      sizeof(rc_config) / sizeof(rc_config[0]);
39
40  const int NUM_SENSORES =
41      sizeof(config) / sizeof(config[0]);
42
43  Boat barco_os(
44      CS_PIN,
45      INTERVALO_LEITURA,
46      LOG_ARQUIVO,
47      config,
48      NUM_SENSORES,
49      rc_config,
50      NUM_CANAIS_RC
51  );
52
53  unsigned long ultimaMedicao = 0;
54
55  #endif // __cplusplus

```

Figura 20 - Ambiente de desenvolvimento platform.io

Para a componente web instalou-se o PHP, o gestor de dependências Composer e a framework Laravel, bem como o servidor MySQL, tendo sido criada a base de dados do projeto e configurado o respetivo acesso no ficheiro de ambiente da aplicação. Por fim, configurou-se o Git e criaram-se dois repositórios no GitHub, um para o firmware do barco (PaP-Hardware-código) e outro para a aplicação web (H2O-Analytics), permitindo manter o histórico de desenvolvimento das duas componentes de forma independente e servir de cópia de segurança do código.

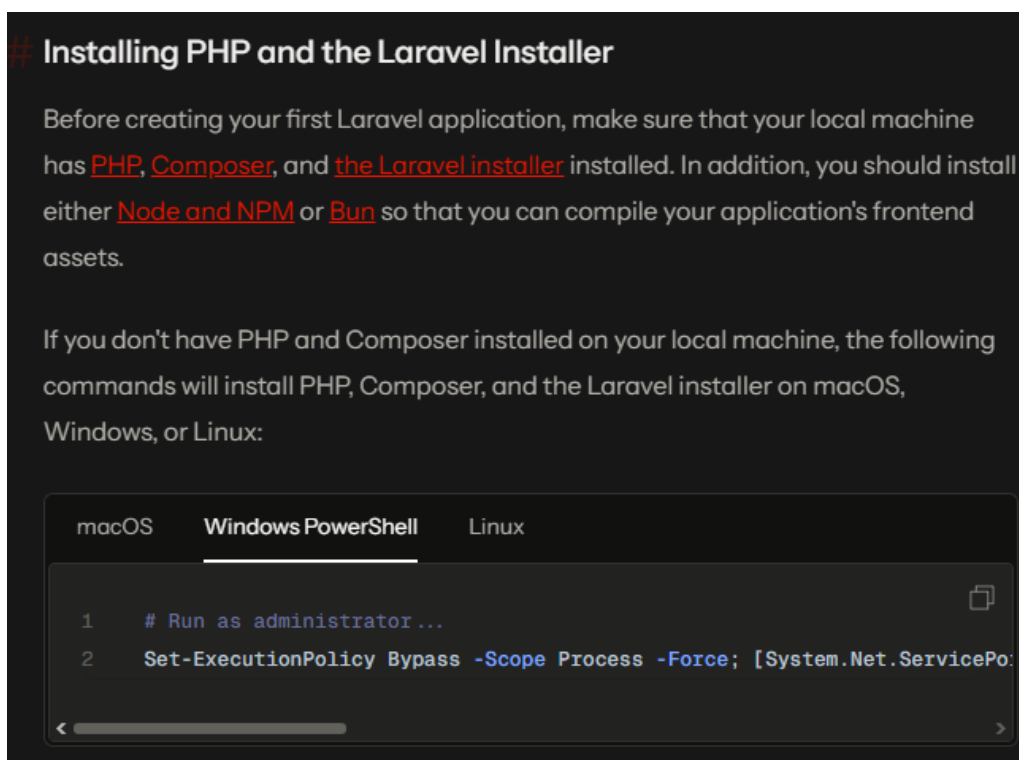


Figura 21 - Instalação do php e laravel

3.2. CONSTRUÇÃO DA EMBARCAÇÃO E PEÇAS 3D

A embarcação assenta num casco de catamarã impresso em 3D, escolhido pela estabilidade que a configuração de casco duplo oferece, característica importante para manter os sensores submersos em posição constante e para acomodar a eletrónica num plano horizontal. Para a instalação das sondas foram abertos furos na lateral do casco, por onde passam os sensores de pH, de condutividade e de temperatura, ficando as sondas em contacto permanente com a água; o sensor de turbidez foi instalado diretamente no casco. Para melhorar a estanquicidade da embarcação foi aplicado um spray impermeabilizante em torno do casco e das zonas de passagem dos cabos. A componente de desenho e impressão 3D foi a única parte do projeto não realizada por mim, por falta de tempo e pela indisponibilidade da impressora 3D da escola, tendo o design sido reutilizado (secção 2.3) e a impressão contratada a uma empresa externa, conforme descrito no capítulo 4.

O casco e as respetivas peças estruturais foram impressos em 3D por uma empresa externa, uma vez que a escola não dispunha de meios para o efeito. As peças foram encomendadas à empresa Impress3D, sediada em Lisboa. Foram encomendadas as seguintes peças: os dois cascos do catamarã, a vara de ligação entre os cascos e os suportes/compartimentos da eletrónica. Alguns dos suportes

do motor chegaram com um diâmetro superior ao do tubo previsto, pelo que foi necessário cortar uma manga de borracha e inseri-la entre os suportes para garantir um encaixe firme. Foi ainda necessário queimar/derreter ligeiramente algumas peças para as adaptar ao design final do projeto.

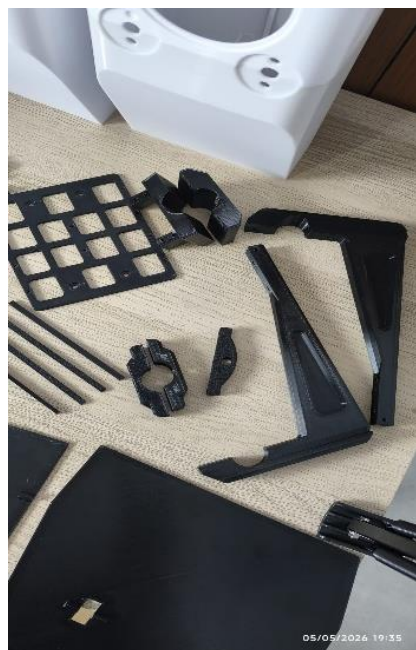


Figura 23 - Peças da impressão 3D



Figura 22 - Casco incompleto da embarcação



Figura 24 - Cascos da embarcação impressos em 3D



Figura 25 - Cascos unidos pela vara de ligação



Figura 26 - Sondas montadas nos furos laterais do casco

3.3. SISTEMA DE PROPULSÃO E ALIMENTAÇÃO

A propulsão é assegurada por um motor brushed fornecido em conjunto com as hélices, comandado por um ESC HobbyWing SeaKing 30A, um variador náutico com refrigeração a água. A direção é feita por um servo que aciona o leme, com o sinal ligado ao pino D8 e alimentação pelo BEC do próprio ESC. Os comandos do operador chegam através do recetor FlySky FS-iA6, de seis canais, emparelhado com o respetivo comando.

A alimentação de todo o sistema provém de uma única bateria NiMH de 7,2 V e 5000 mAh, com uma divisão em dois ramos: o ramo de potência alimenta diretamente o ESC e o motor, enquanto o ramo lógico passa por um conversor step-down que reduz a tensão para 5 V estáveis, alimentando o Arduino, os sensores e os módulos. Esta separação reduz o impacto do ruído elétrico e das quedas de tensão provocadas pelo motor nas leituras analógicas dos sensores.

Durante os testes de propulsão surgiu um problema relevante com o corte por baixa tensão (LVC) do ESC, cuja proteção vinha configurada de fábrica para baterias LiPo e interpretava a curva de descarga da NiMH como bateria fraca, cortando o motor. O problema, descrito em detalhe no capítulo 4, foi resolvido reprogramando o ESC através do próprio comando, ajustando o limiar de corte ao tipo de bateria utilizado. Após esta correção, o sistema de propulsão e direção foi testado com sucesso em água, tendo o barco navegado de forma controlada.



Figura 27 - Montagem final da embarcação H2O-Maré

3.4. MONTAGEM DO CIRCUITO ELETRÓNICO

3.4.1. CONTROLO DA EMBARCAÇÃO E CONFIGURAÇÃO DO COMANDO

A embarcação é conduzida manualmente através de um comando de radiocontrolo FlySky FS-i6, emparelhado com o recetor FlySky FS-ia6 de seis canais instalado a bordo. O recetor comunica com os atuadores e com o Arduino Mega, que lê o estado de cada canal nas entradas A8 a A13 como sinais PWM.

No comando foram configurados os canais de acordo com as funções da embarcação: o canal do acelerador (stick direito, eixo vertical) comanda o ESC e, através dele, a velocidade do motor; o canal da direção (stick direito, eixo horizontal) comanda o servo do leme. O canal 3 foi atribuído à aceleração do motor para a frente e o canal 4 ao movimento lateral do barco. O canal 6 está ligado ao pino A5 do Arduino através de um potenciómetro (knob): quando o valor lido ultrapassa 1700, o firmware dispara uma medição, servindo assim como interruptor auxiliar. Não foi necessário inverter nenhum canal; a única configuração adicional foi a reprogramação do ESC através do próprio comando RC. Foi ainda efetuada a calibração dos cursos (end points) e o emparelhamento (binding) entre o comando e o recetor antes dos testes em água.



Figura 28 - Recetor FlySky FS-iA6

A montagem eletrónica seguiu o esquema elétrico apresentado na Figura 19, distribuindo os periféricos pelos diferentes barramentos do Arduino Mega de forma a evitar conflitos de comunicação. Os quatro sensores analógicos foram ligados às entradas A0 a A4, cada um através da sua placa de condicionamento de sinal, que converte a grandeza medida numa tensão entre 0 e 5 V, legível pelo conversor analógico-digital do microcontrolador. O sensor de temperatura DS18B20 foi ligado ao pino A3 em modo digital, com a resistência de pull-up de 4,7 k Ω exigida pelo protocolo 1-Wire.

O leitor de cartão MicroSD utiliza o barramento SPI (pinos D50 a D53), o RTC DS3231 o barramento I2C (D20 e D21) e o módulo SIM808 uma porta série por hardware a 38 400 baud. Os seis canais do recetor FlySky são lidos nas entradas A8 a A13 como sinais PWM. A tabela seguinte resume o mapa de ligações.

Componente	Ligação ao Arduino Mega
Cartão MicroSD	SPI — MISO D50, MOSI D51, SCK D52, CS D53
RTC DS3231	I2C — SDA D20, SCL D21
Sensor de turbidez (SEN0189)	Entrada analógica A0
Sensor de EC (DFR0300)	Entrada analógica A1
Sensor de DO (SEN0237)	Entrada analógica A2
Sensor de temperatura (DS18B20)	A3, protocolo 1-Wire com pull-up de 4,7 k Ω a 5 V
Sensor de pH (SEN0161-V2)	Entrada analógica A4
Módulo SIM808	Serial1 (D18/D19) a 38 400 baud
Recetor FlySky FS-iA6	Canais CH1–CH6 em A8–A13 (leitura PWM por pulseIn)
Servo de direção	Sinal em D8, alimentado pelo BEC do ESC

Tabela 4 - Ligações ao Arduino Mega

À data da redação deste relatório, a cablagem definitiva encontra-se em fase de conclusão, estando as ligações validadas em protótipo de bancada.

3.5. DESENVOLVIMENTO DO FIRMWARE

BIBLIOTECAS UTILIZADAS

O firmware apoia-se num conjunto de bibliotecas externas, declaradas no ficheiro platformio.ini e geridas automaticamente pelo PlatformIO. São elas: ArduinoJson (versão 7), para construir e serializar cada registo em formato JSON; SD, para a escrita no cartão MicroSD; DFRobot_PH e DFRobot_EC10, que implementam a calibração e a conversão dos sensores de pH e de condutividade; OneWire e DallasTemperature, para a leitura do sensor de temperatura DS18B20; RTCLib, para o relógio de tempo real DS3231; a biblioteca SIM808-EVB, para a comunicação com o módulo SIM808 por comandos AT; AltSoftSerial, para uma porta série adicional por software; e elapsedMillis, para a temporização não-bloqueante das leituras.

3.5.1. ARQUITETURA DO CÓDIGO

O firmware foi desenvolvido em C++ no PlatformIO, com uma arquitetura orientada a objetos organizada em bibliotecas próprias, cada uma responsável por um subsistema: Sensor (leitura e calibração dos sensores), GpsController (comunicação com o SIM808), Relogio (RTC DS3231), SdFileStorage (escrita no cartão), FlyskyController (leitura dos canais RC) e Debug (diagnóstico do sistema). Uma classe central, Boat, compõe todas estas classes e orquestra o funcionamento do sistema, mantendo o ficheiro principal reduzido à configuração.

A configuração do hardware é orientada a tabelas: os sensores e os canais RC são declarados em arrays de estruturas no ficheiro principal, indicando o nome, o pino, o tipo e o estado de cada um. Esta abordagem torna o sistema extensível, adicionar um sensor ou mudar um pino resume-se a alterar uma linha do array, sem tocar na lógica das classes.

```
Sensor::sensor_info config[] = {
    { "Turbidez",          A0, TURBIDEZ,    INATIVO },
    { "EC Sensor",         A1, EC_SENSOR,   INATIVO },
    { "Dissolved Oxygen Sensor", A2, DO_SENSOR, INATIVO },
    { "Temperatura",       A3, TEMPERATURA, INATIVO },
    { "Ph Sensor",         A4, Ph_SENSOR,   INATIVO },
};
```

```

const int CS_PIN = 53;

static constexpr unsigned long INTERVALO_LEITURA = 5000;

const char* LOG_ARQUIVO = "dados.txt";

FlyskyController::rc_info rc_config[] = {
    { "CH6", A8, RC_ATIVO },
    { "CH2", A9, RC_INATIVO },
    { "CH3", A10, RC_INATIVO },
    { "CH4", A11, RC_INATIVO },
    { "CH5", A12, RC_INATIVO },
    { "CH1", A13, RC_INATIVO },
};

Sensor::sensor_info config[] = {
    { "Dissolved Oxygen Sensor", A0, DO_SENSOR, ATIVO },
    { "EC Sensor", A1, EC_SENSOR, ATIVO },
    { "Ph Sensor", A2, Ph_SENSOR, ATIVO },
    { "Temperatura", A3, TEMPERATURA, ATIVO },
    { "Turbidez", A4, TURBIDEZ, ATIVO },
};

const int NUM_CANAIS_RC =
    sizeof(rc_config) / sizeof(rc_config[0]);

const int NUM_SENSORES =
    sizeof(config) / sizeof(config[0]);

```

Figura 29 - Excerto de código: configuração dos sensores e dos canais RC

No arranque, o sistema apresenta um menu pela porta série com três modos de operação: «calibrar», que executa as rotinas de calibração dos sensores de pH e de EC; «debug», que imprime o estado de todos os subsistemas e o mapa de pinos esperado, facilitando o diagnóstico da cablagem em campo; e «run», o modo normal de recolha. Se nenhum comando for recebido em cinco segundos, o sistema entra automaticamente em modo de recolha — comportamento essencial para o arranque autónomo a bordo, sem computador ligado.

3.5.2. AQUISIÇÃO DAS LEITURAS E CALIBRAÇÃO

A calibração dos sensores é um passo essencial para garantir leituras fiáveis e foi implementada diretamente no firmware, acessível pelo modo «calibrar» do menu de arranque. Os sensores de pH e de condutividade são calibrados com o apoio das bibliotecas oficiais da DFRobot (DFRobot_PH e DFRobot_EC10): o sensor de pH é calibrado por dois pontos, mergulhando a sonda em soluções tampão de pH 4,0 e pH 7,0; o sensor de condutividade é calibrado com uma solução padrão de condutividade conhecida. O sensor de oxigénio dissolvido é calibrado por um ponto único, à saturação no ar, servindo essa leitura de referência para a tabela de saturação usada na conversão. A calibração da turbidez foi confirmada antes dos testes de recolha. Por falta de tempo, não foi possível

confirmar a calibração do sensor de temperatura, ficando esta verificação pendente para trabalho futuro.

Quanto aos cuidados a ter com os sensores: as sondas de pH e de oxigénio dissolvido são frágeis e não podem secar — devem ser guardadas com a solução de conservação apropriada —; todos os sensores devem ser enxaguados com água destilada entre medições para evitar contaminação cruzada; e as leituras de condutividade, de oxigénio e de pH são compensadas em temperatura, pelo que o correto funcionamento do sensor DS18B20 é crítico, como se descreve mais à frente

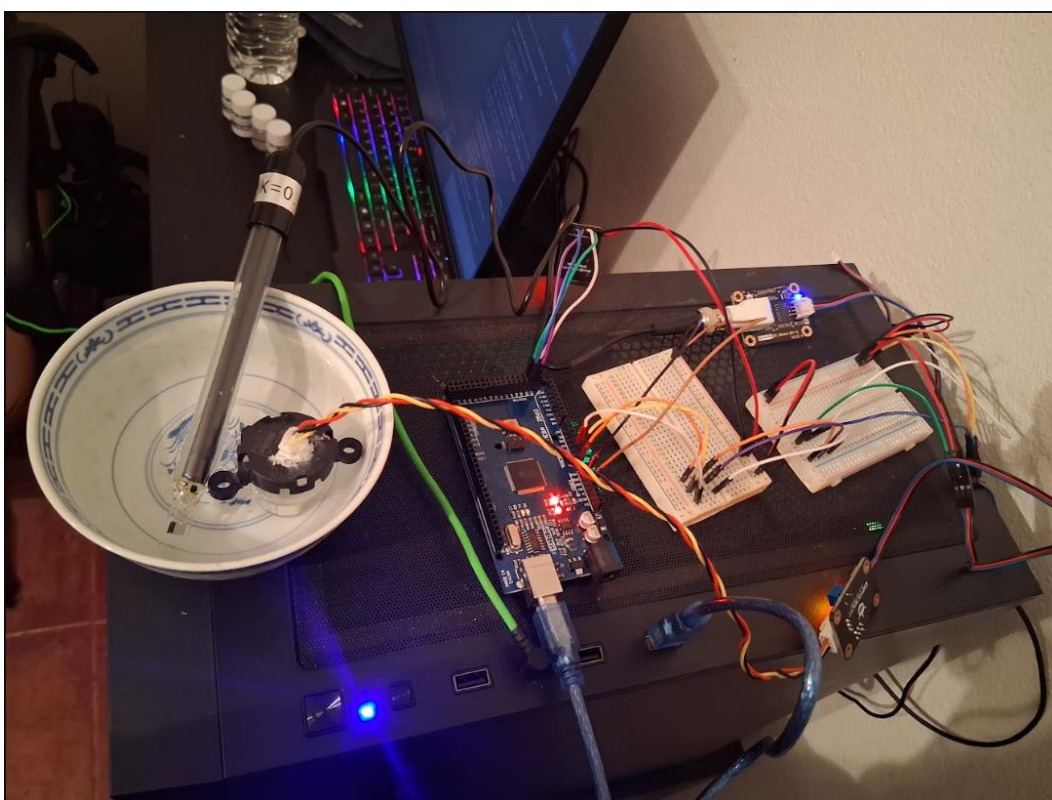


Figura 30 - Configuração dos sensores

. Em modo de recolha, o programa lê os sensores e converte os valores em bruto do conversor analógico-digital nas unidades físicas de cada parâmetro. As conversões de pH e de condutividade utilizam as bibliotecas oficiais DFRobot_PH e DFRobot_EC10, que aplicam as curvas de calibração guardadas na memória; o oxigénio dissolvido é calculado através de uma tabela de saturação de 41 entradas (0 a 40 °C), com calibração de ponto único à saturação a 25 °C. Nos três casos é aplicada a compensação de temperatura a partir da leitura do DS18B20, uma vez que a condutividade, a solubilidade do oxigénio e a resposta do elétrodo de pH variam com a temperatura da água.

A dependência de todos os cálculos em relação ao sensor de temperatura revelou-se um ponto crítico: quando o DS18B20 falhou durante o desenvolvimento (ver capítulo 4), todos os parâmetros

compensados devolveram valores errados em cascata. A solução foi o método `getTemperaturaSegura()`, que valida cada leitura de temperatura e, em caso de falha do sensor, utiliza a última leitura válida conhecida ou, na sua ausência, o valor de referência de 25 °C — garantindo que uma falha num sensor degrada a precisão, mas não invalida todo o registo.

```
float Sensor::getTemperaturaSegura()
{
    if (findSensorByType(TEMPERATURA) >= 0) {
        float t = lerTemperatura(configs[findSensorByType(TEMPERATURA)].pin);
        if (t > -126.0) {
            this->temperatura = t;
            return t;
        }
    }
    // DS18B20 desligado - usar ultima leitura valida ou 25 graus C
    if (this->temperatura <= -126.0) this->temperatura = 25.0;
    return this->temperatura;
}
```

Figura 23 — Excerto de código: leitura segura da temperatura (fallback)

3.5.3. GEORREFERENCIAÇÃO COM O SIM808

A colocação do módulo SIM808 a funcionar exigiu várias configurações. Ao contrário de um módulo GPS que emite continuamente tramas NMEA, o SIM808 é controlado por comandos AT enviados por uma porta série por hardware a 38 400 baud: no arranque, o firmware liga a alimentação do módulo, ativa o subsistema GNSS (comando `AT+CGNSPWR=1`) e passa a pedir periodicamente a posição (`AT+CGNSINF`), interpretando a resposta. Foi necessário aguardar a fixação de satélites — mais demorada no primeiro arranque (cold start) — e validar essa fixação antes de aceitar qualquer coordenada, bem como converter as coordenadas recebidas para graus decimais com o sinal correto.

```

bool GpsController::aguardarFix(unsigned long timeoutMs) {
    if (!moduloDisponivel_) {
        Serial.println(F("[GPS] Modulo indisponivel - a saltar espera por FIX."));
        return false;
    }

    Serial.println(F("[GPS] A aguardar FIX dos satelites..."));

    unsigned long inicio = millis();
    unsigned long ultimoPrint = 0;
    const unsigned long INTERVALO_PRINT = 2000;

    while (millis() - inicio < timeoutMs) {
        if (sim808_.getGnssFixStatus()) {
            fixObtido_ = true;
            Serial.println(F("[GPS] FIX obtido."));
            return true;
        }

        if (millis() - ultimoPrint >= INTERVALO_PRINT) {
            uint8_t visiveis = 0, usados = 0;
            obterSatelites(visiveis, usados);
            Serial.print(F("[GPS] Sem FIX ainda - satelites visiveis: "));
            Serial.print(visiveis);
            Serial.print(F(", usados: "));
            Serial.println(usados);
            ultimoPrint = millis();
        }
    }

    Serial.println(F("[GPS] Timeout a espera de FIX - a continuar sem GPS."));
    return false;
}

```

Figura 31 - GPS controller

As coordenadas geográficas são obtidas através do módulo SIM808, controlado por comandos AT numa porta série por hardware a 38 400 baud, com recurso à biblioteca SIM808-EVB. No arranque, o firmware liga o subsistema GNSS do módulo e aguarda a fixação de satélites; em cada leitura, converte as coordenadas recebidas — em graus, minutos e segundos — para graus decimais com o respetivo sinal, o formato utilizado pela plataforma web. Quando ainda não há fixação, os campos de latitude e longitude são simplesmente omitidos do registo, e a plataforma associa a medição às coordenadas da localização escolhida pelo utilizador no momento do carregamento — uma decisão de desenho que evita gravar coordenadas inválidas.

A escolha do SIM808 tem ainda uma vantagem estratégica: além do GNSS, o módulo integra um modem GSM/GPRS, o que significa que a transmissão de dados em tempo real pela rede móvel — identificada como melhoria futura — poderá ser implementada sem adicionar hardware ao barco.

3.5.4. REGISTO TEMPORAL E GRAVAÇÃO DOS DADOS

A cada registo é associada a data e a hora fornecidas pelo RTC DS3231, formatadas como texto no padrão dia-mês-ano-hora-minuto-segundo; se o RTC tiver perdido a alimentação, o firmware acerta-o automaticamente com a hora de compilação do programa. O registo completo — carimbo temporal, valores dos sensores agrupados num objeto e coordenadas quando disponíveis — é construído com a biblioteca ArduinoJson e serializado como uma linha JSON acrescentada a um ficheiro de texto no cartão MicroSD. Uma vez que o Arduino não dispõe de suporte nativo para ficheiros .json, o ficheiro tem extensão .txt, sendo a interpretação efetuada pela aplicação web no momento da importação. Por decisão de desenho, é criado um ficheiro novo em cada sessão de recolha; se a escrita no cartão falhar, a gravação é suspensa e o erro é sinalizado, evitando registos corrompidos.

```
JsonDocument Boat::coletarDados()
{
    JsonDocument doc;
    doc["timestamp"] = meu_relogio.obterDataFormatada();

    Coordenadas coords = gps.obterCoordenadas();
    if (coords.valido) {
        doc["latitude"] = coords.latitude;
        doc["longitude"] = coords.longitude;
    }

    JsonObject sensoresObj = doc["sensores"].to<JsonObject>();
    for (int i = 0; i < sensores.getQuantidade(); i++) {
        sensoresObj[sensores.getNome(i)] = sensores.readValue(i);
    }

    return doc;
}
```

Figura 24 — Excerto de código: construção do registo JSON

```
{"timestamp":"14-05-2026-10-32-00","latitude":39.406700,"longitude":-
9.133900,"sensores":{"Turbidez":2.81,"EC Sensor":512.4,"Dissolved Oxygen
Sensor":8210,"Temperatura":18.6,
"Ph Sensor":7.2}}
```

```

GPSTEST.TXT
Ficheiro  Editar  Ver  H1  B  I  S  E  D  A

{"timestamp": "05-07-2026-00-00-00", "latitude": 39.3377849, "longitude": -9.1390229, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-05", "latitude": 39.3377867, "longitude": -9.1389307, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-10", "latitude": 39.3377998, "longitude": -9.1389507, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-15", "latitude": 39.3378157, "longitude": -9.138958, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-20", "latitude": 39.3378106, "longitude": -9.1389775, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-25", "latitude": 39.3378122, "longitude": -9.1390146, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-30", "latitude": 39.3378203, "longitude": -9.1390463, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-35", "latitude": 39.3378114, "longitude": -9.1390764, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-40", "latitude": 39.3378116, "longitude": -9.1390839, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-45", "latitude": 39.3378041, "longitude": -9.1391105, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-50", "latitude": 39.3377838, "longitude": -9.1391064, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-00-55", "latitude": 39.3377858, "longitude": -9.1391085, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-00", "latitude": 39.3377659, "longitude": -9.1391134, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-05", "latitude": 39.33776, "longitude": -9.1390859, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-10", "latitude": 39.3377484, "longitude": -9.1390865, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-15", "latitude": 39.3377528, "longitude": -9.1390449, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-20", "latitude": 39.3377531, "longitude": -9.1390192, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-25", "latitude": 39.3377586, "longitude": -9.1389986, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-30", "latitude": 39.3377552, "longitude": -9.1389773, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-35", "latitude": 39.3377631, "longitude": -9.1389461, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-40", "latitude": 39.337771, "longitude": -9.1389304, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-45", "latitude": 39.3377715, "longitude": -9.1389262, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-50", "latitude": 39.3377864, "longitude": -9.1389373, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-01-55", "latitude": 39.3377954, "longitude": -9.1389486, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-00", "latitude": 39.3378049, "longitude": -9.138963, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-05", "latitude": 39.3378131, "longitude": -9.1389714, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-10", "latitude": 39.3378144, "longitude": -9.1390024, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-15", "latitude": 39.3378287, "longitude": -9.1390317, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-20", "latitude": 39.3378209, "longitude": -9.1390495, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-25", "latitude": 39.3378182, "longitude": -9.1390846, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-30", "latitude": 39.3378047, "longitude": -9.1391043, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-35", "latitude": 39.3378003, "longitude": -9.1390983, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-40", "latitude": 39.3377908, "longitude": -9.1391143, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-45", "latitude": 39.337782, "longitude": -9.1391057, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-50", "latitude": 39.3377589, "longitude": -9.1391103, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-02-55", "latitude": 39.3377549, "longitude": -9.1390917, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-03-00", "latitude": 39.3377609, "longitude": -9.1390721, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-03-05", "latitude": 39.3377462, "longitude": -9.1390311, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-03-10", "latitude": 39.3377473, "longitude": -9.1390092, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-03-15", "latitude": 39.3377511, "longitude": -9.138977, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-03-20", "latitude": 39.3377527, "longitude": -9.138969, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-03-25", "latitude": 39.3377608, "longitude": -9.138944, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}
{"timestamp": "05-07-2026-00-03-30", "latitude": 39.3377817, "longitude": -9.138932, "sensores": {"turbidez": 123, "ac": 456, "do": 789, "temperatura": 25.3, "ph": 7.1}}

```

Figura 32 - Exemplo de uma linha do ficheiro de dados da piscina

3.6. DESENVOLVIMENTO DA APLICAÇÃO WEB

3.6.1. CONFIGURAÇÃO INICIAL E BASE DE DADOS

ESTRUTURA DA APLICAÇÃO: ROTAS, CONTROLADORES E VISTAS

A aplicação segue o padrão Model-View-Controller (MVC) da framework Laravel. O ficheiro de rotas define os pontos de entrada da aplicação e associa cada um a um controlador, protegendo as áreas reservadas com middleware de autenticação e de perfil. As rotas públicas incluem o mapa principal e a página de detalhe de cada localização; as rotas de colaborador incluem o carregamento de ficheiros e a listagem dos próprios envios; e as rotas de administração, agrupadas sob o prefixo «admin», incluem o painel, a moderação de medições, a gestão de utilizadores, de localizações, de mapeamentos de sensores e dos pedidos de colaborador.

Do lado dos controladores, o MapController e o LocationController tratam da visualização pública; o UploadController trata da importação dos ficheiros do barco; e os controladores do espaço de administração (DashboardController, MeasurementApprovalController, UserManagementController, LocationManagementController, SensorMappingController e CollaboratorRequestReviewController) implementam a gestão da plataforma. As operações de CRUD (criar, ler, atualizar e eliminar) sobre as localizações e sobre os mapeamentos de sensores foram implementadas com controladores de recurso (resource controllers), que geram automaticamente o conjunto completo de rotas e métodos. Existe ainda uma pequena API interna (LocationGeoJsonController e LocationPointsController) que fornece os dados ao mapa em formato GeoJSON.

A base de dados é criada e versionada através de migrations — uma por tabela: users (com uma migration adicional para o campo role), locations, measurements (com uma migration adicional para os campos de rejeição), sensor_mappings e collaborator_requests, além das tabelas de suporte da framework. As vistas (views) foram construídas com o motor de templates Blade e organizadas por área: autenticação, mapa, detalhe de localização, carregamento, os meus envios, perfil e o conjunto de vistas da administração (painel, moderação, utilizadores, localizações, mapeamentos e pedidos de colaborador).

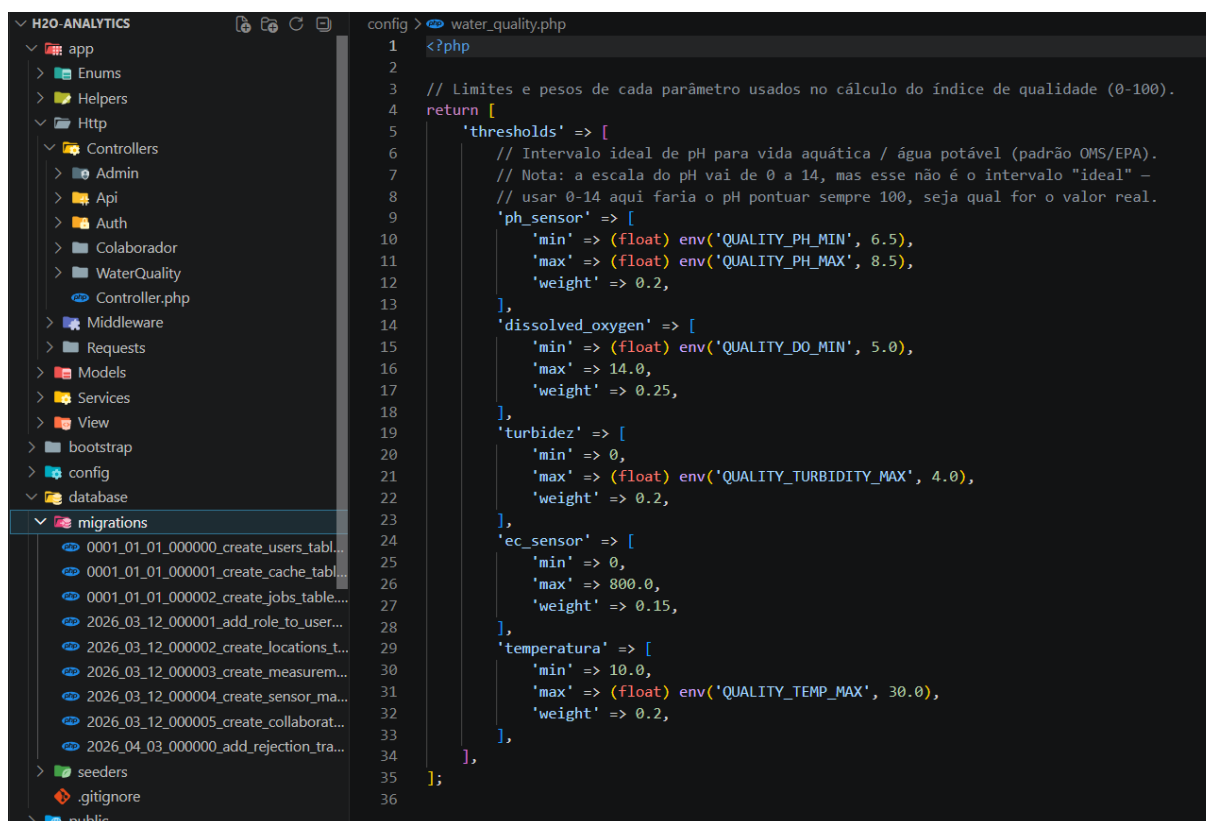


Figura 33 - Exemplo da Estrutura Laravel

A aplicação web foi criada com a framework Laravel, tendo sido configurada a ligação à base de dados MySQL e definido o esquema descrito na secção 2.4 através de migrations — classes PHP que criam e versionam as tabelas, permitindo reconstruir a base de dados em qualquer ambiente com um único comando. Para o desenvolvimento e os testes foram criados seeders que geram dados iniciais (utilizadores de teste com os três perfis e os mapeamentos de sensores), possibilitando testar o mapa e as listagens sem depender de dados reais do barco.

3.6.2. AUTENTICAÇÃO E PERFIS DE UTILIZADOR

A autenticação foi implementada com o pacote Laravel Breeze, que fornece o registo, o início de sessão e a recuperação de palavra-passe. Sobre esta base foi implementado um sistema de três perfis através do campo role da tabela users — utilizador autenticado, colaborador e administrador —, aplicado por um middleware próprio que protege as rotas: apenas colaboradores acedem às páginas de carregamento de ficheiros e apenas administradores acedem à área de administração. Os utilizadores podem solicitar o estatuto de colaborador através de um pedido registado na tabela collaborator_requests, que o administrador aprova ou rejeita.

3.6.3. IMPORTAÇÃO E VALIDAÇÃO DOS DADOS DO BARCO

O fluxo de entrada de dados começa com o carregamento do ficheiro gerado pelo barco, associado a uma localização escolhida pelo colaborador. A aplicação interpreta o conteúdo como JSON, aceitando tanto um único registo como uma lista, valida a presença do carimbo temporal e do bloco de sensores, e converte o carimbo para tempo Unix através de uma classe auxiliar que aceita tanto números como o formato de texto produzido pelo RTC. Os nomes dos sensores são normalizados pelo serviço SensorParser, que consulta a tabela sensor_mappings e traduz os aliases vindos do firmware para os nomes canónicos da base de dados, descartando valores não numéricos.

Cada registo é gravado com o método firstOrCreate sobre o par localização/carimbo temporal, o que garante a deduplicação: se o mesmo ficheiro for carregado duas vezes, os registos repetidos são ignorados em vez de duplicados. Quando o registo não traz coordenadas, por falta de fixação GPS no momento da recolha, são usadas as coordenadas da localização escolhida. Todas as medições entram com o estado pendente, sujeitas ao fluxo de moderação.

```
$measurement = Measurement::firstOrCreate(
    [
        'location_id' => $location->id,
        'timestamp_sensor' => $ts,
    ],
    [
        'user_id' => $user->id,
        'latitude' => $entry['latitude'] ?? $location->latitude,
        'longitude' => $entry['longitude'] ?? $location->longitude,
        'raw_data' => $entry,
        // ... parametros normalizados pelo SensorParser ...
        'status' => MeasurementStatus::Pending->value,
    ]
);
```

Figura 26 — Excerto de código: importação com deduplicação (firstOrCreate)

3.6.4. ÍNDICE DE QUALIDADE DA ÁGUA

Para tornar os dados compreensíveis para o público não técnico, a plataforma calcula um índice de qualidade da água de 0 a 100 para cada localização. Cada parâmetro tem um intervalo ideal e um peso definidos num ficheiro de configuração: dentro do intervalo, o parâmetro pontua 100; fora dele, a pontuação diminui proporcionalmente à distância. O índice global é a média ponderada das

pontuações, sendo depois traduzido numa classificação qualitativa de Excelente, Bom, Razoável, Mau ou Muito Mau e numa cor, usada diretamente nos marcadores do mapa.

Parâmetro	Intervalo ideal	Peso
pH	0,0 – 14,0	20 %
Oxigénio dissolvido	5,0 – 14,0 mg/L	25 %
Turbidez	0 – 4,0	20 %
Condutividade elétrica	0 – 800 μ S/cm	15 %
Temperatura	10 – 30 °C	20 %

Tabela 5 - Parâmetros, intervalos ideais e pesos do índice de qualidade

Os intervalos ideais de cada parâmetro foram definidos a partir de valores de referência para a qualidade da água em corpos de água doce (água potável e ecossistemas aquáticos de água doce), com base no Decreto-Lei n.º 236/98, de 1 de agosto, que estabelece as normas, critérios e objetivos de qualidade da água em Portugal, nomeadamente para águas doces superficiais destinadas a fins aquícolas (águas piscícolas). Os pesos refletem a importância relativa atribuída a cada parâmetro para a saúde de um ecossistema aquático, tendo o oxigénio dissolvido o maior peso (25 %) por ser determinante para a vida aquática. Estes limites e pesos estão centralizados num ficheiro de configuração da aplicação, o que permite ajustá-los sem alterar o código.

3.6.5. MAPA INTERATIVO E VISUALIZAÇÃO

Em termos técnicos, a coloração dos marcadores é conseguida da seguinte forma: uma API interna devolve, para cada localização, as médias das medições aprovadas em formato GeoJSON; no navegador, a biblioteca Leaflet.js desenha um marcador circular por localização e o índice de qualidade calculado no servidor determina a cor aplicada a cada marcador (verde a vermelho, do Excelente ao Muito Mau). Ao clicar num marcador, é apresentada uma janela com a classificação, o número de medições e as médias dos cinco parâmetros.

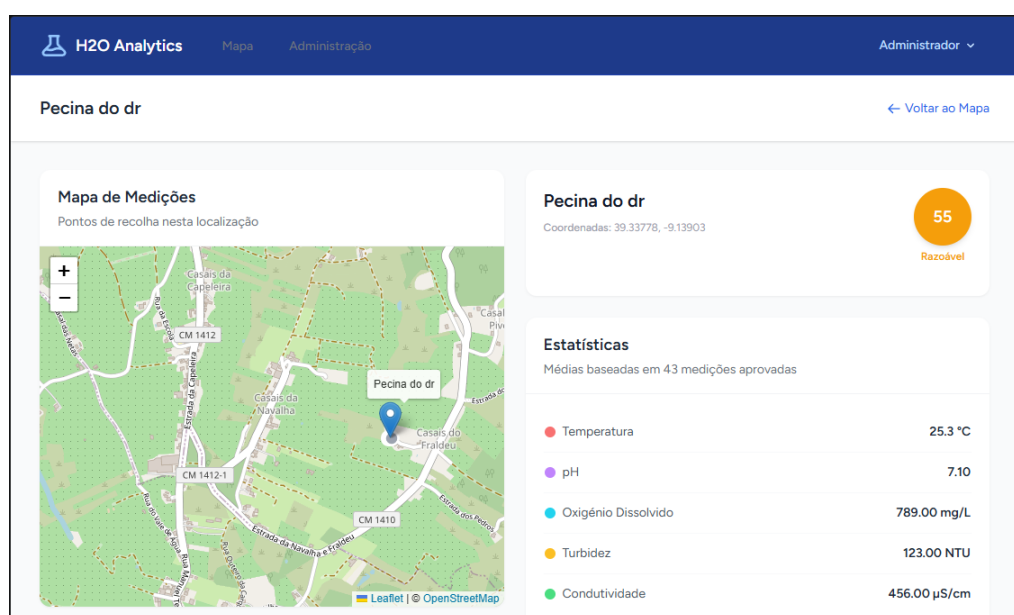


Figura 34 - Exemplo Mapa de visualização de dados

A visualização assenta num mapa interativo construído com a biblioteca Leaflet.js, alimentado por uma API interna que devolve as localizações em formato GeoJSON com as médias das medições aprovadas. Cada localização é apresentada como um marcador circular cuja cor reflete o índice de qualidade; ao clicar, um popup mostra a classificação, o número de medições aprovadas e as médias dos cinco parâmetros com as respetivas unidades. A página de detalhe de cada localização apresenta um mini-mapa com os pontos de recolha individuais, cada um com o autor e o instante da medição e um gráfico temporal, construído com Chart.js, que permite analisar a evolução dos parâmetros ao longo do tempo.

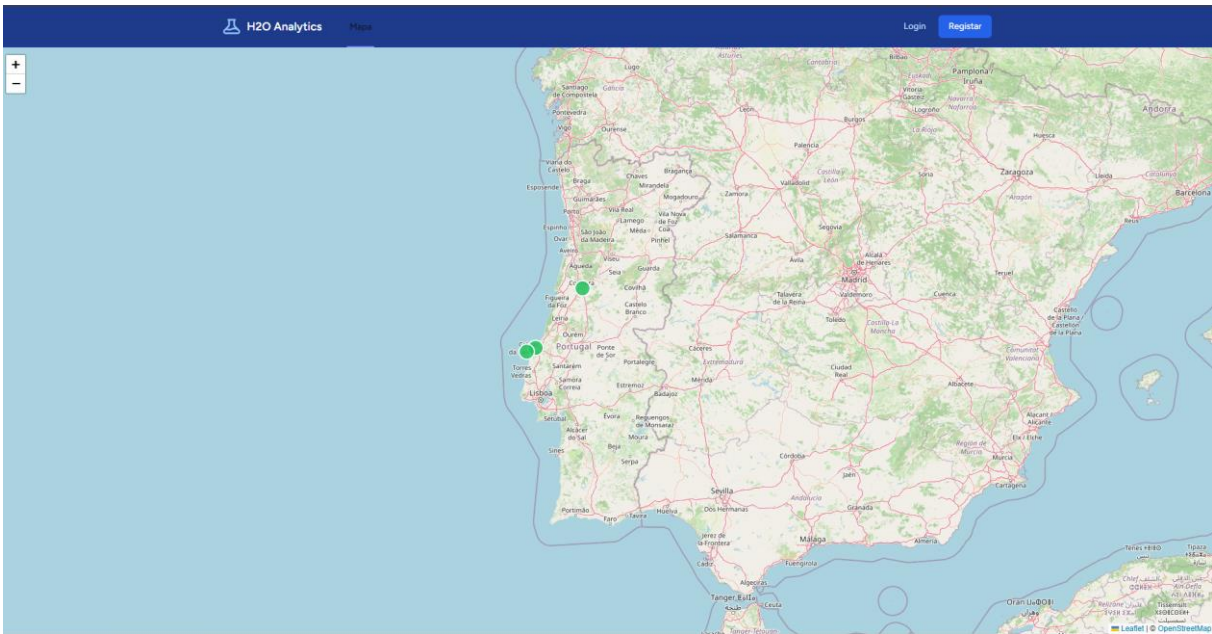


Figura 35 - Screenshot do mapa com marcadores coloridos pelo índice de qualidade

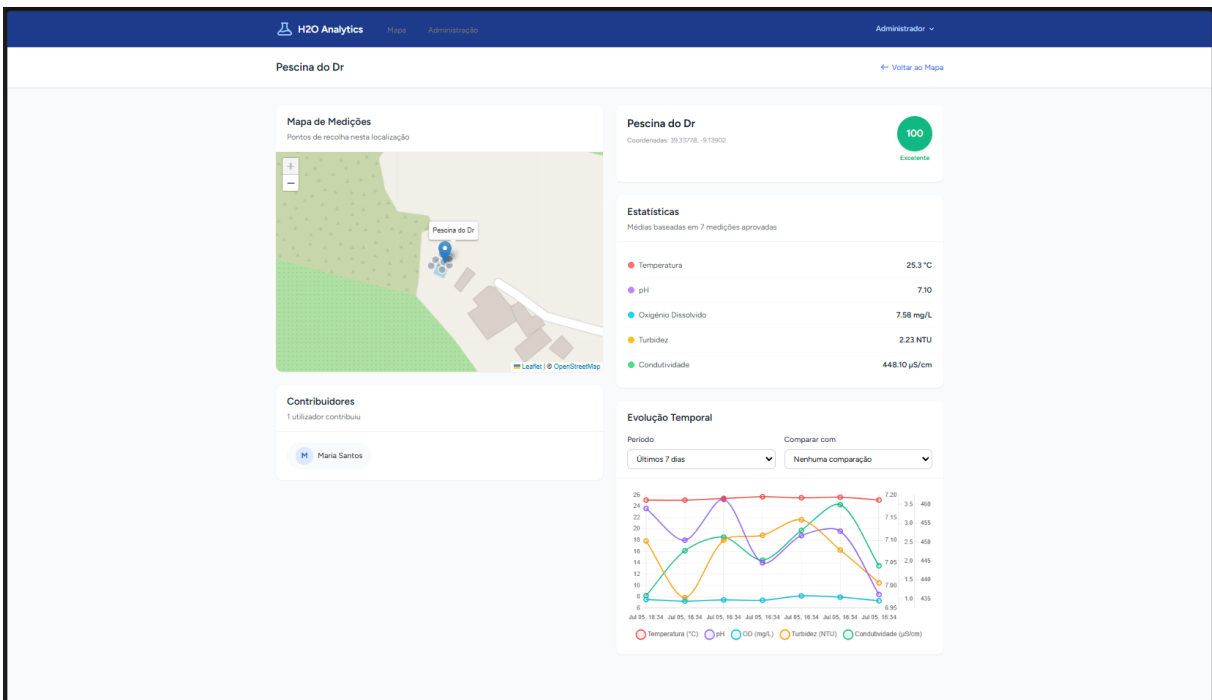
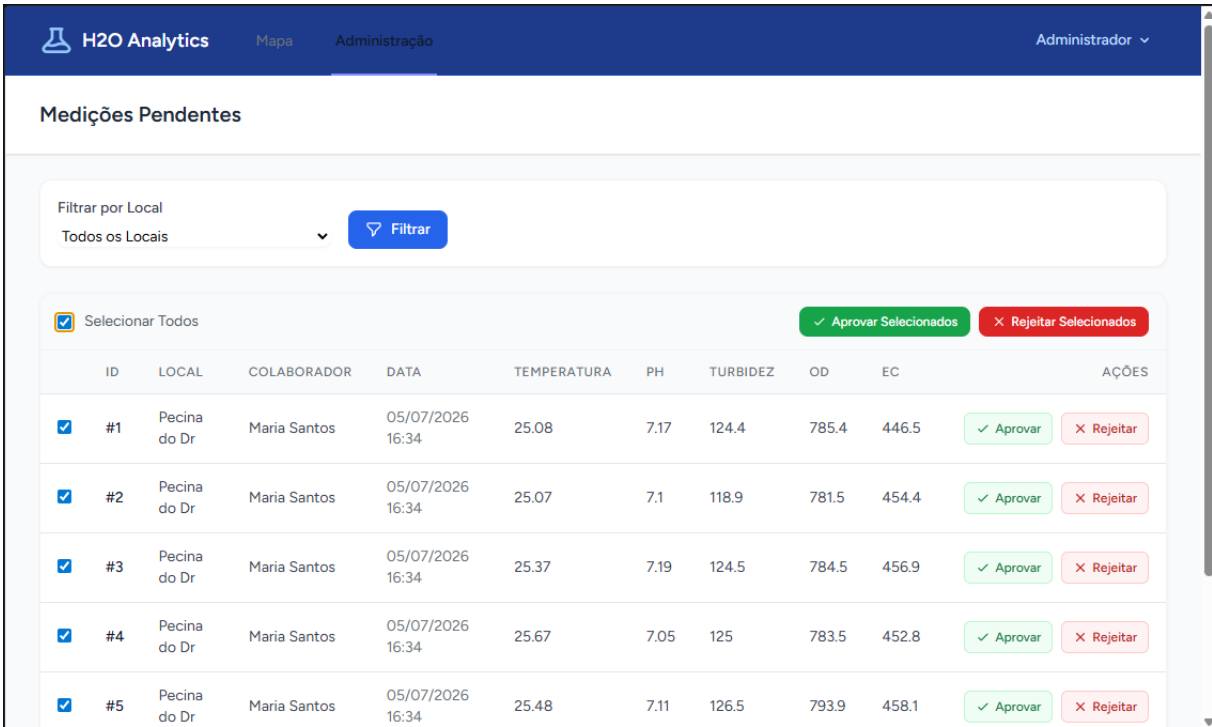


Figura 36 - Screenshot da página de detalhe de uma localização (gráfico temporal)

3.6.6. ÁREA DE ADMINISTRAÇÃO

A área de administração, acessível apenas ao perfil de administrador, concentra a gestão da plataforma: um painel inicial com estatísticas, a moderação das medições — com aprovação e rejeição individuais ou em massa, registando quem moderou e quando —, a gestão de utilizadores e dos seus perfis, a gestão das localizações, a revisão dos pedidos de colaborador e a manutenção dos mapeamentos de sensores, que permitem adaptar a plataforma a alterações na nomenclatura do firmware sem tocar no código.



The screenshot shows the 'H2O Analytics' administration interface. The top navigation bar includes 'Mapa' and 'Administração', with the user role 'Administrador' selected. The main section is titled 'Medições Pendentes'. It features a filter dropdown set to 'Todos os Locais' and a 'Filtrar' button. Below the filter, there are buttons for 'Selecionar Todos', 'Aprovar Selecionados', and 'Rejeitar Selecionados'. The main content is a table with the following columns: ID, LOCAL, COLABORADOR, DATA, TEMPERATURA, PH, TURBIDEZ, OD, EC, and AÇÕES. Five rows of data are displayed, each with a selection checkbox and 'Aprovar'/'Rejeitar' buttons.

ID	LOCAL	COLABORADOR	DATA	TEMPERATURA	PH	TURBIDEZ	OD	EC	AÇÕES
☒ #1	Pecina do Dr	Maria Santos	05/07/2026 16:34	25.08	7.17	124.4	785.4	446.5	<input checked="" type="button" value="Aprovar"/> <input checked="" type="button" value="Rejeitar"/>
☒ #2	Pecina do Dr	Maria Santos	05/07/2026 16:34	25.07	7.1	118.9	781.5	454.4	<input checked="" type="button" value="Aprovar"/> <input checked="" type="button" value="Rejeitar"/>
☒ #3	Pecina do Dr	Maria Santos	05/07/2026 16:34	25.37	7.19	124.5	784.5	456.9	<input checked="" type="button" value="Aprovar"/> <input checked="" type="button" value="Rejeitar"/>
☒ #4	Pecina do Dr	Maria Santos	05/07/2026 16:34	25.67	7.05	125	783.5	452.8	<input checked="" type="button" value="Aprovar"/> <input checked="" type="button" value="Rejeitar"/>
☒ #5	Pecina do Dr	Maria Santos	05/07/2026 16:34	25.48	7.11	126.5	793.9	458.1	<input checked="" type="button" value="Aprovar"/> <input checked="" type="button" value="Rejeitar"/>

Figura 37 - Screenshot da área de administração (aprovação de medições)

3.7. INTEGRAÇÃO E TESTES

A integração das duas componentes foi validada de ponta a ponta com o fluxo completo de dados: recolha das leituras pelo barco, gravação no cartão MicroSD, exportação do ficheiro, carregamento na plataforma web, moderação e apresentação dos pontos no mapa. Para acelerar a validação foi criado um programa de teste dedicado que grava no cartão registos com o formato real do sistema, coordenadas verdadeiras do SIM808 e valores de sensores fixos, permitindo testar a importação na plataforma antes de a calibração de todos os sensores estar concluída.

O sistema de propulsão e direção foi testado em água, confirmando-se a navegação controlada via RC, registada em vídeo. Durante os testes foram identificadas e corrigidas falhas menores, sobretudo na validação dos dados importados e na apresentação dos marcadores no mapa.



Figura 38 - Teste de navegação na água

Importa referir, com transparência, o estado atual e as limitações do sistema: a cablagem definitiva e a calibração fina de alguns sensores encontram-se em fase de conclusão; o ciclo automático de recolha a bordo, incluindo o disparo de uma medição através de um canal do comando RC, está desenhado, mas ainda em implementação; a transferência de dados exige a remoção física do cartão MicroSD; e a navegação é exclusivamente manual. Estas limitações foram identificadas desde o início como aceitáveis para esta fase do projeto e constam das melhorias futuras apresentadas na conclusão.

4. PROBLEMAS E SOLUÇÕES ENCONTRADAS

Ao longo do desenvolvimento surgiram vários problemas, primeiro na construção do protótipo e, depois, na componente eletrónica, que exigiram diagnóstico e a procura de soluções. Descrevem-se em seguida os mais relevantes, pela ordem em que foram enfrentados.

O primeiro grande obstáculo foi financeiro: o orçamento do projeto não foi aprovado, visto que ultrapassava a verba disponibilizada pela escola para cada projeto pelo que a aquisição uma boa parte do material teve de ser suportada por investimento privado (neste caso familiar). Esta limitação condicionou as escolhas de componentes e obrigou a uma gestão cuidadosa de cada compra ao longo de todo o projeto, levando mais tempo a ser coordenado e finalizado

A esta dificuldade somou-se o longo tempo de espera pelo material encomendado, várias semanas, no caso dos sensores vindos de fornecedores internacionais e ainda a receção de material trocado, que teve de ser devolvido e substituído, o motor foi um dos que veio diferente e também o esc.

Para não bloquear o desenvolvimento, foram utilizados componentes alternativos temporários como, por exemplo, o servo, a bateria e o motor, que permitiram desenvolver e testar o código enquanto os componentes definitivos não chegavam.

Na construção do protótipo, a escola não dispunha de meios para a impressão 3D do casco, pelo que foi necessário procurar ajuda externa, uma procura dificultada pelo orçamento reduzido do projeto. A empresa contratada acabou por entregar uma impressão rudimentar: algumas peças vieram com defeitos, outras em quantidade inferior à encomendada, e faltavam mesmo algumas peças do design original, o que tornou a montagem particularmente difícil. As peças foram adaptadas manualmente, com recurso a cola e a pequenos cortes e ajustes, até garantirem o encaixe correto na embarcação.



Figura 39 - Montagem das peças impressas em 3D



Figura 40 - Colagem das peças em 3D

Resolvidos os problemas de construção, seguiram-se os desafios da componente eletrónica. Um dos problemas mais instrutivos foi a falha do sensor de temperatura DS18B20. Como as conversões de EC, de DO e de pH dependem da compensação térmica, a falha de um único sensor propagou-se em cascata e invalidou os cálculos de todos os parâmetros compensados. O diagnóstico foi facilitado pelo valor sentinela devolvido pela biblioteca (-127 °C), que identifica um sensor desligado. A solução teve duas frentes: no hardware, a verificação da cablagem e da resistência de pull-up do barramento 1-Wire; no firmware, a criação do método `getTemperaturaSegura()`, que valida cada leitura e recorre à última temperatura válida — ou a 25 °C como valor de referência — quando o sensor falha, tornando o sistema resiliente a falhas futuras.

No sistema de propulsão, o ESC HobbyWing SeaKing 30A entrava em corte inesperado durante os testes: tratava-se do mecanismo de proteção por baixa tensão (LVC), calibrado de fábrica para baterias LiPO, que interpretava a curva de descarga da bateria NiMH, de tensão nominal mais baixa, como uma situação de bateria fraca e cortava a alimentação do motor. O diagnóstico exigiu a análise do comportamento da tensão da bateria em carga, e a solução foi reprogramar o ESC através do próprio comando RC, ajustando o limiar de corte ao tipo de bateria utilizado.

Na georreferenciação, a abordagem inicial previa a utilização da biblioteca TinyGPS++, padrão para módulos GPS que transmitem tramas NMEA continuamente. Contudo, o módulo utilizado é um SIM808, que funciona por comandos AT, o pedido de coordenadas tem de ser feito explicitamente, e a resposta vem num formato próprio, não como fluxo NMEA, pelo que a TinyGPS++ nunca chegou a receber dados válidos. Após o diagnóstico, a solução foi adotar a biblioteca SIM808-EVB, específica para este módulo, e implementar na classe `GpsController` a conversão das coordenadas de graus, minutos e segundos para graus decimais, bem como a validação da fixação de satélites antes de aceitar qualquer leitura.

5. CONCLUSÃO

Chegado ao final do desenvolvimento do projeto, importa confrontar os resultados obtidos com os objetivos definidos na introdução.

O primeiro objetivo consistia na construção e teste de um barco de rádio controlo equipado com sensores e um módulo GPS. Este objetivo foi alcançado, uma vez que a embarcação navega de forma controlada através de rádio controlo, conforme comprovado nos testes realizados em ambiente real. Os cinco sensores encontram-se instalados no barco, estando apenas a cablagem definitiva e a calibração de alguns sensores em fase de conclusão.

O segundo objetivo consistia na implementação de um sistema de registo de dados georreferenciados num cartão MicroSD. Este objetivo foi igualmente cumprido, sendo que cada registo associa os parâmetros medidos às coordenadas geográficas obtidas pelo módulo SIM808 e ao respetivo carimbo temporal fornecido pelo módulo RTC, armazenando toda a informação em formato JSON.

O terceiro objetivo consistia no desenvolvimento de uma plataforma web recorrendo à framework Laravel e à base de dados MySQL. Este objetivo não só foi alcançado como também superado, uma vez que a plataforma disponibiliza um mapa interativo, um índice de qualidade da água, um sistema de autenticação com três níveis de acesso, um mecanismo de moderação das medições e uma área de administração completa.

O quarto objetivo incidia na integração dos conhecimentos adquiridos ao longo do curso e pode considerar-se plenamente concretizado. O desenvolvimento do projeto exigiu a aplicação de competências nas áreas da programação em C++ e PHP, eletrónica, bases de dados e desenvolvimento web, permitindo consolidar e aplicar, de forma prática, os conhecimentos adquiridos durante a formação.

Apesar dos resultados alcançados, o projeto apresenta algumas limitações. O processo automático de recolha de dados a bordo encontra-se concebido, mas ainda em fase de implementação. Atualmente, a transferência dos dados é efetuada através da remoção física do cartão MicroSD e a navegação da embarcação continua a ser exclusivamente manual.

Como trabalho futuro, prevê-se a conclusão da cablagem e da calibração de todos os sensores, a implementação do acionamento das medições através de um canal do comando de rádio controlo, a transmissão dos dados em tempo real através da rede móvel, recorrendo ao modem GSM/GPRS já integrado no módulo SIM808, a aplicação de técnicas de filtragem e cálculo de médias para aumentar a estabilidade das leituras, a conversão dos valores de turbidez para unidades NTU, a implementação

de navegação autónoma com percursos previamente definidos e o aperfeiçoamento da interface de consulta e filtragem dos dados disponibilizados na plataforma web.

De uma forma geral, considera-se que o projeto demonstra ser possível desenvolver um sistema funcional de monitorização ambiental georreferenciada utilizando hardware de baixo custo e software de código aberto. Os resultados obtidos validam o conceito inicialmente proposto e demonstram o potencial da solução para futuras aplicações em contexto real, nomeadamente por associações ambientais, instituições de ensino ou entidades locais responsáveis pela monitorização da qualidade da água.

Autoavaliação

Globalmente, o balanço do trabalho desenvolvido é muito positivo. Este constituiu o projeto mais completo e complexo realizado ao longo do percurso formativo, por exigir a integração de áreas que habitualmente são desenvolvidas de forma independente, nomeadamente eletrónica, programação de sistemas embarcados, desenvolvimento web full-stack e bases de dados.

Um dos aspetos mais relevantes foi a conceção da arquitetura do firmware, organizada em classes reutilizáveis e configurável através de tabelas, bem como a capacidade de ultrapassar diversos desafios relacionados com o hardware. Entre estes destacam-se o problema do corte por baixa tensão do controlador eletrónico de velocidade (ESC) e a falha em cascata provocada pelo sensor de temperatura, situações que exigiram um processo de diagnóstico rigoroso e a identificação das respetivas causas, em vez da simples aplicação de soluções previamente existentes.

Caso o projeto fosse desenvolvido novamente, seria dada maior prioridade à aquisição antecipada dos componentes, à verificação precoce da compatibilidade entre as bibliotecas de software e os módulos utilizados, uma vez que a incompatibilidade entre a biblioteca TinyGPS++ e o módulo SIM808 originou atrasos significativos. Seria igualmente reservado mais tempo para a calibração dos sensores, uma fase que se revelou mais exigente do que o inicialmente previsto.

Apesar destes aspetos, considera-se que a estratégia adotada foi adequada. A divisão do desenvolvimento em duas vertentes paralelas, firmware e plataforma web, permitiu manter a evolução do projeto sempre que uma das componentes ficava temporariamente condicionada pela indisponibilidade de material. Da mesma forma, a decisão de reutilizar um modelo tridimensional existente para o casco, em vez de desenvolver um novo de raiz, revelou-se fundamental para otimizar o tempo disponível e garantir a conclusão dos objetivos essenciais dentro do prazo estabelecido.

REFERÊNCIAS BIBLIOGRÁFICAS

- Arduino. (s.d.). SD Library. Arduino Documentation. <https://docs.arduino.cc/libraries/sd/>
- Blanchon, B. (s.d.). ArduinoJson Documentation. <https://arduinojson.org/>
- DFRobot. (s.d.). Gravity: Analog Dissolved Oxygen Sensor. DFRobot Wiki.
https://wiki.dfrobot.com/Gravity__Analog_Dissolved_Oxygen_Sensor_SKU_SEN0237
- DFRobot. (s.d.). Gravity: Analog Electrical Conductivity Sensor (K=1). DFRobot Wiki.
https://wiki.dfrobot.com/Gravity__Analog_Electrical_Conductivity_Sensor___Meter_V2__K%3D1__SKU_DFR0300
- DFRobot. (s.d.). Gravity: Analog pH Sensor / Meter Kit V2. DFRobot Wiki.
https://wiki.dfrobot.com/Gravity__Analog_pH_Sensor_Meter_Kit_V2_SKU_SEN0161-V2
- DFRobot. (s.d.). Turbidity Sensor SKU SEN0189. DFRobot Wiki.
https://wiki.dfrobot.com/Turbidity_sensor_SKU__SEN0189
- Diário da República. (1998, agosto 1). Decreto-Lei n.º 236/98. Diário da República n.º 176/1998, Série I-A. <https://diariodarepublica.pt/dr/detalhe/decreto-lei/236-1998-430457>
- DroneBot Workshop. (2021, agosto 25). [RC Robot Car - RC Controls and Arduino] [Vídeo]. YouTube. <https://youtu.be/BACBNgaCnJU>
- electrosync. (2020, outubro 14). 3D Printed RC Utility Boat — Hull and Drive System — Part 1 [Vídeo]. YouTube. <https://youtu.be/0KblE-0bh7o>
- Laravel. (s.d.). Laravel Documentation. <https://laravel.com/docs>
- Leaflet. (s.d.). Leaflet — an open-source JavaScript library for interactive maps. <https://leafletjs.com/>
- PlatformIO. (s.d.). PlatformIO Documentation. <https://docs.platformio.org/>
- Ramos, G. [gabs Ramos Hobbyist]. (2019, abril 9). [SIM808 GPS / GSM Locator with Arduino] [Vídeo]. YouTube. <https://youtu.be/DeW1LN4BdYU>
- Tropratik. (s.d.). SIM808-EVB — Arduino library for the SIM808 module. GitHub.
<https://github.com/Tropratik/SIM808-EVB>

IDENTIFICAÇÃO DO ALUNO

NOME COMPLETO: TIAGO MIGUEL SANTOS SILVA

N.º DE PROCESSO: 105687

CICLO DE ESTUDOS: 12.º ANO

TELEMÓVEL: 925798617

E-MAIL: TIAGOMIGUEL042006@GMAIL.COM
